Mini Project Report

On

"TrustFi: AI-Assisted and Privacy-Aware Digital Lending Platform"

By

VIDHI JAGLAGANESHWALA - 1272250479

For

MBC40130 Mini Project

SY MSc Blockchain Techonology

AY 2026-2027

Under The Guidance Of

Mr. Sudhanshu Jaiswal

&

Mr. Kaustubh Keer

Submitted To





In partial fulfillment of the requirement for the

award of Degree of

Master of Science (Blockchain Technology)

Submitted Through

MIT-WPU

School of Computer Science and Engineering

Department of Computer Science and Applications

Dr. Vishwanath Karad
MIT WORLD PEACE UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

MIT-WPU
॥ विश्वशान्तिर्ध्रुवं ध्रुवा ॥

Department of Computer Science and Applications

Certificate

This is to certify that, VIDHI JAGLAGANESHWALA student of **SY MSc BT, Semester III** has completed Mini Project entitled TrustFi: AI-Assisted and Privacy-Aware Digital Lending Platform under guidance of,

Mr. Sudhanshu Jaiswal & Mr. Kaustubh Keer

in partial fulfilment of requirement for the award of **Master of Science Blockchain Technology** prescribed under

Dr. Vishwanath Karad MIT World Peace University, Pune

This project is the record of authentic work carried out by him / her out during the academic year 2026-2027.

Mr. Sudhanshu Jaiswal
Internal Guide

Mr. Kaustubh Keer
Internal Guide

Dr. Gopal Sakarkar
Program Head- MSc BT

Dr. Anuradha S. Kanade
Program Director

Date: ___/___/_____

Acknowledgement

I wish to express our profound gratitude to all the individuals who supported us throughout the completion of our Mini Project III. This report is a testament to their guidance, patience, and resources.

We are deeply indebted to our esteemed project guide, **Mr. Sudhanshu Jaiswal & Mr. Kaustubh Keer**, for their invaluable mentorship, direction, and encouragement. Their advice was crucial in defining the scope and methodology of this project.

We extend our sincere thanks to the **Head of the Program, Dr. Gopal Sakarkar**, and the **Program Director, Dr. Anuradha S. Kanade**, for providing the necessary facilities, academic framework, and environment conducive to advanced project work and research.

Our appreciation also goes to all the **Faculty Members** of the **Department of Computer Science and Applications** for their comprehensive instruction and expertise, which equipped us with the knowledge foundational to this project.

We thank our fellow **Project Group Members,** for their unwavering dedication, collaborative spirit, and commitment to shared goals. This project is a product of our collective effort.

Finally, we are grateful to our **Classmates** for their constructive feedback and discussions, which enriched our understanding and helped us refine our approach.

Name:	Vidhi Jaglaganeshwala
PRN:	1272250479
Place:	MIT-WPU,PUNE

Date: ___/___/_____

Declaration

I, Vidhi Jaglaganeshwala , on behalf of my team here by declare that this Mini Project report entitled **"TrustFi: AI-Assisted and Privacy-Aware Digital Lending Platform"** is an authentic record of our own work carried out under the supervision of **Mr. Sudhanshu Jaiswal & Mr. Kaustubh Keer**.

I further declare that the content of this report, including all design, coding, testing, and analysis, is original and has not been submitted previously for any other degree or diploma at this or any other institution.

We take sole and full responsibility for any errors, omissions, or misrepresentations found within this report.

Signature:	
Name:	Vidhi Jaglaganeshwala
PRN:	1272250479
Place:	MIT-WPU,PUNE

Project Report Index

Index	Section Title
1	**INTRODUCTION**
1.1	EXISTING SYSTEM, PROPOSED SYSTEM AND NEED AND OBJECTIVE OF SYSTEM
1.2	SCOPE OF WORK
1.3	OPERATING ENVIRONMENT – HARDWARE AND SOFTWARE
1.4	DESCRIPTION OF TECHNOLOGY USED
2	**PROPOSED SYSTEM**
2.1	FEASIBILITY STUDY
2.2	USER REQUIREMENTS
3	**ANALYSIS AND DESIGN**
3.1	OBJECT DIAGRAM
3.2	CLASS DIAGRAM
3.3	USE CASE DIAGRAM
3.4	SEQUENCE DIAGRAM
3.5	ACTIVITY DIAGRAM
3.6	MODULE HIERARCHY DIAGRAM
3.7	USER INTERFACE DESIGN (SCREENS ETC.)
3.8	TABLE SPECIFICATIONS
4	**INPUT AND OUTPUT SCREENS**
5	**TEST PROCEDURES AND IMPLEMENTATION**
6	**SAMPLE CODE**
7	**LIMITATIONS**
8	**FUTURE SCOPE**
9	**CONCLUSION**
10	**REFERENCES**

1. INTRODUCTION

TrustFi is an AI-assisted and privacy-aware digital lending platform designed to simplify and organize the loan application process through web-based technologies. The system aims to provide a centralized platform where applicants can submit loan applications, upload supporting documents, track application status, and access relevant loan-related services. It also provides administrative functionality for reviewing applications and managing loan decisions according to the configured workflow.

Traditional loan application processes often involve physical documentation, repeated data entry, manual verification, and communication between applicants and lending personnel. These activities may increase processing time and make application tracking difficult. TrustFi aims to address these challenges by integrating digital application management, document processing, authentication, and configurable risk-assessment logic into a unified platform.

The system is designed to support property loans and gold loans. Applicants can provide personal, employment, financial, and loan-related information and upload the required supporting documents. The platform includes OCR-related functionality using Tesseract OCR to support information extraction from suitable documents. Extracted information and submitted application details can be processed through configured validation and assessment logic.

TrustFi also incorporates loan eligibility evaluation and fraud-risk assessment functionality. These mechanisms are intended to assist the application review process by identifying relevant information, potential inconsistencies, and configured risk indicators. The assessment results should be treated as decision-support information rather than an automatic guarantee of loan approval or fraud detection.

The platform follows a web-based architecture consisting of a frontend developed using Next.js, React, and TypeScript, and a backend developed using Node.js, Express.js, and TypeScript. REST APIs support communication between the frontend and backend, while Prisma ORM provides database access. JSON Web Tokens (JWT) are used for authentication. Razorpay-related payment functionality is included in the project context, with its operational status dependent on the implemented integration and configuration.

Privacy-aware lending is an important consideration in the design of TrustFi because loan applications may contain sensitive identity and financial information. The project explores privacy-preserving concepts, including blockchain and Fully Homomorphic Encryption (FHE). However, these concepts should be described as implemented features only where the source code and configuration demonstrate their actual operation. Any functionality that remains conceptual or incomplete is considered a potential future enhancement.

Overall, TrustFi aims to demonstrate how modern web development, document processing, automated assessment logic, and secure application workflows can be combined to support a more organized digital lending process. The project provides an academic implementation of a lending platform while recognizing the importance of data privacy, reliable verification, transparent assessment, and human review in financial decision-making.

1.1 EXISTING SYSTEM, PROPOSED SYSTEM AND NEED AND OBJECTIVE OF SYSTEM

Existing System

In a conventional lending process, applicants may need to visit a lending institution, complete physical or digital forms, submit identity and financial documents, and communicate with loan officers during verification and approval. Depending on the institution, different stages of the process may be managed through separate systems or manual procedures.

Document verification and information extraction can require considerable manual effort. Applicants may also find it difficult to obtain timely updates about their application status. Where digital systems are available, the level of automation, integration, and tracking may vary between institutions.

The following limitations may occur in traditional or partially digitized lending workflows:

1. Dependence on manual data entry and document handling.
2. Repeated submission or verification of applicant information.
3. Delays in processing and reviewing applications.
4. Limited visibility into application progress.
5. Difficulty identifying inconsistencies across submitted documents.
6. Limited integration between document processing, eligibility evaluation, and application management.

These limitations vary across lending institutions and should not be interpreted as applying to every existing lending system.

Proposed System

TrustFi proposes a centralized web-based platform for managing loan applications and supporting the review process. It combines applicant-facing functionality with administrative application management, document upload, OCR-based information extraction, and configured eligibility and fraud-risk assessment logic.

The proposed system includes the following capabilities:

1. User registration and login with authentication.
2. Submission and management of property loan and gold loan applications.
3. Uploading of identity, financial, and other supporting documents.
4. OCR-assisted extraction of information from supported documents.
5. Configured validation checks for submitted information and documents.
6. Loan eligibility assessment based on the available application data and implemented rules.
7. Fraud-risk evaluation using configured indicators and assessment logic.

8. Application-status tracking for applicants.
9. Administrative review and management of loan decisions.
10. Payment-related functionality where the integration is implemented and configured.

The proposed system is intended to improve organization, reduce repetitive manual effort where automation is available, and provide a structured workflow for applicants and administrators. Final lending decisions should remain subject to appropriate verification, applicable policies, and authorized review.

Need of the System

A digital lending platform requires a structured method for collecting application information, handling supporting documents, performing preliminary assessments, and tracking application progress. TrustFi addresses this need by bringing these activities together within a single application workflow.

The need for TrustFi arises from the following considerations:

- **Digital accessibility:** Applicants need a convenient way to submit loan applications through a web interface.
- **Document processing:** OCR can assist in extracting relevant information from supported documents and reduce repetitive data entry.
- **Structured assessment:** Configured eligibility and fraud-risk logic can help organize information for further review.
- **Application tracking:** A centralized status workflow can improve visibility into application progress.
- **Administrative efficiency:** Authorized personnel need a structured interface for reviewing submitted applications and recording decisions.
- **Security and privacy:** Identity and financial information require appropriate authentication, access control, and secure data handling.
- **Extensibility:** A modular architecture can support future improvements in verification, risk assessment, reporting, and privacy-preserving computation.

Objectives of the System

The main objectives of TrustFi are:

1. To develop a web-based platform for digital loan application management.
2. To support property loan and gold loan application workflows.
3. To enable applicants to submit relevant personal, employment, financial, and loan-related information.
4. To provide document-upload functionality for supporting loan applications.
5. To use OCR technology to assist with extracting information from supported documents.

6. To implement configured validation and loan eligibility assessment logic.
7. To evaluate available fraud-risk indicators to support application review.
8. To provide application-status tracking and administrative review functionality.
9. To integrate authentication and database access using the selected technologies.
10. To support payment-related workflows where implemented and configured.
11. To examine privacy-aware approaches to processing sensitive lending information.
12. To design the platform so that additional verification and assessment capabilities can be introduced in future versions.

1.2 SCOPE OF WORK

The scope of TrustFi is to develop a web-based digital lending platform that organizes the loan application process and supports document handling, preliminary assessment, and administrative review. The platform focuses on property loans and gold loans and provides separate functionality for applicants and authorized administrators.

The project covers the following areas:

1. Loan Application Management

The system is designed to allow applicants to enter personal, employment, financial, and loan-related details and submit applications for supported loan categories. Application information is managed through the application's configured workflow.

2. Document Upload and Processing

Applicants can upload supporting documents required for loan applications. The project includes OCR-related functionality using Tesseract OCR to assist in extracting information from supported document formats. The accuracy of extraction depends on document quality, layout, language, and the implemented processing pipeline.

3. Document Validation

The platform aims to support validation of submitted information and documents through configured checks. Such checks can help identify missing information or inconsistencies but do not independently establish the authenticity of every document.

4. Loan Eligibility Assessment

TrustFi includes eligibility-assessment logic intended to evaluate application information against configured criteria. The assessment supports preliminary evaluation and should not be considered a substitute for a lending institution's complete verification and approval process.

5. Fraud-Risk Assessment

The system includes fraud-risk assessment functionality intended to examine available indicators and produce risk-related information. Its effectiveness depends on the implemented rules, input data, and validation. A risk score or indicator alone does not prove fraudulent activity.

6. Property and Gold Loan Workflows

The platform is intended to support applications for property loans and gold loans. Each category may require different information and supporting documents according to the configured application workflow.

7. Authentication and Authorization

The project uses JWT-based authentication to support authenticated access. Access to protected operations should be controlled according to the application's implemented authorization rules.

8. Administrative Review

Authorized administrators can use the available application-management functionality to review applications and manage decisions according to the implemented workflow.

9. Application Status Tracking

The platform aims to provide applicants with visibility into the progress of their loan applications through the status information maintained by the system.

10. Payment-Related Functionality

The project includes a Razorpay-related integration in its technology context. The scope of operational payment processing depends on the implemented integration, valid configuration, and successful testing.

11. Privacy-Aware Design

The project considers the importance of protecting sensitive financial and identity information. Blockchain and Fully Homomorphic Encryption (FHE) are relevant privacy-related concepts explored in the project context. Any operational implementation of these technologies must be verified from the source code before being claimed as a completed feature.

12. Technology Integration

The platform combines a Next.js and React frontend with a Node.js and Express.js backend written in TypeScript. REST APIs support frontend-backend communication, while Prisma ORM is used for database access.

The scope of this academic project is limited to the functionality available in the current implementation. Features that are incomplete, unconfigured, or conceptual are treated as limitations or future enhancements rather than completed capabilities.

1.3 OPERATING ENVIRONMENT – HARDWARE AND SOFTWARE

TrustFi is designed as a web-based application with a frontend, backend, and database-access layer. Its operating environment consists of the software tools and hardware resources required to develop, configure, run, and test these components.

1.3.1 Hardware Requirements

The following table provides suggested hardware requirements for local development and testing. These are indicative requirements and must be adjusted according to the actual development environment.

Sr. No.	Hardware Component	Suggested Requirement
1	Processor	Intel Core i5 or equivalent processor
2	RAM	8 GB minimum; 16 GB recommended
3	Storage	At least 10 GB of available storage for project files and dependencies
4	Display	Standard monitor or laptop display
5	Input Devices	Keyboard and mouse or laptop touchpad
6	Network	Internet connection for installing dependencies and accessing configured external services

These specifications are recommendations for development and testing, not verified minimum requirements for production deployment.

1.3.2 Software Requirements

Sr. No.	Software / Technology	Purpose
1	Operating System	Windows, Linux, or macOS compatible with the development tools
2	Next.js	Frontend application framework
3	React	Component-based user interface development
4	TypeScript	Type-safe development for frontend and backend code
5	Node.js	JavaScript runtime for the backend and development tools
6	Express.js	Backend application and REST API development
7	Zustand	Frontend application state management
8	Prisma ORM	Database access and data-model management
9	Database Provider	The provider configured in the project's Prisma schema and environment

10	Tesseract OCR	Optical character recognition for supported documents
11	JSON Web Tokens (JWT)	Authentication token handling
12	Razorpay	Payment integration, subject to implementation and configuration
13	Web Browser	Accessing and testing the web application
14	Code Editor / IDE	Editing and maintaining project source code
15	Git	Version control, if configured and used by the project

1.3.3 Execution Environment

The frontend is developed using Next.js, React, and TypeScript. It communicates with the Express.js backend through REST APIs. The backend processes application requests, performs configured business logic, and uses Prisma ORM to interact with the configured database.

Tesseract OCR is used for supported document-processing tasks where the relevant processing pipeline is implemented and configured. Authentication relies on JWT-related functionality, including appropriate backend secret configuration. Payment-related operations require the corresponding Razorpay configuration and integration.

The application can be developed and tested locally using a compatible Node.js environment and web browser. The exact runtime versions, database provider, API endpoints, and environment variables should be documented from the project's configuration before final submission.

1.4 DESCRIPTION OF TECHNOLOGY USED

TrustFi uses modern web development technologies to provide a structured digital lending workflow. The technology stack includes frontend development tools, backend technologies, database access, document processing, authentication, and external service integration. Each technology serves a specific purpose within the application architecture.

1.4.1 Next.js

Next.js is a React-based web framework used to develop the frontend of TrustFi. It provides routing and application-structuring capabilities that help organize the user interface into different pages and components. In the lending platform, the frontend is responsible for presenting application forms, document-upload interfaces, application details, and other user-facing features implemented in the project.

1.4.2 React

React is a JavaScript library for building user interfaces through reusable components. It enables the development of interactive interfaces by dividing the application into smaller and maintainable UI

elements. In TrustFi, React supports the frontend components used for displaying forms, application information, status details, and administrative interfaces where implemented.

1.4.3 TypeScript

TypeScript extends JavaScript with static type-checking capabilities. It helps developers define data types, identify certain programming errors during development, and maintain consistency across application components. TrustFi uses TypeScript in its frontend and backend code to improve code organization and maintainability.

1.4.4 Node.js

Node.js is a JavaScript runtime that allows JavaScript and TypeScript-based backend applications to execute outside a web browser. It provides the execution environment for the TrustFi backend and supports request handling, application logic, and integration with supporting libraries.

1.4.5 Express.js

Express.js is a web application framework for Node.js. It simplifies the development of HTTP routes, middleware, and REST APIs. In TrustFi, Express.js is used to implement backend endpoints that process frontend requests, manage application operations, and connect the user interface with the backend business logic.

1.4.6 REST APIs

Representational State Transfer (REST) is an architectural style commonly used for communication between web applications and backend services. REST APIs enable a client to send requests to a server and receive responses using HTTP methods such as GET, POST, PUT, and DELETE, depending on the endpoint design.

TrustFi uses REST API communication between its frontend and backend. These APIs support the application's configured operations, such as authentication, loan application handling, document-related requests, and administrative workflows.

1.4.7 Zustand

Zustand is a lightweight state-management library for React applications. It allows shared application state to be maintained through stores and accessed by relevant components. In TrustFi, Zustand is used for frontend state management where configured, helping coordinate information between components without requiring all state to be passed through multiple component levels.

1.4.8 Prisma ORM

Prisma is an Object-Relational Mapping (ORM) tool that provides a structured interface for accessing supported databases. It allows developers to define data models and perform database operations using application code. TrustFi uses Prisma ORM as its database-access layer.

The actual database provider is determined by the project's Prisma schema and configuration. The database name, tables, fields, and relationships should therefore be documented only after examining the current schema and migration files.

1.4.9 Tesseract OCR

Tesseract OCR is an optical character recognition engine that can extract machine-readable text from supported images and document inputs. OCR can assist in converting scanned or photographed documents into text for further processing.

In TrustFi, Tesseract OCR supports the document-processing workflow where the relevant functionality is implemented. The extracted text may be used by configured information-extraction or validation logic. OCR output can contain errors, especially when documents are unclear, rotated, poorly illuminated, or use unsupported layouts or languages. Consequently, extracted information should be checked before it is relied upon for important lending decisions.

1.4.10 JSON Web Tokens (JWT)

JSON Web Token (JWT) is a token format commonly used to represent claims between systems. In web applications, tokens can support authenticated requests when they are generated, validated, and handled securely.

TrustFi uses JWT-related authentication functionality to support protected application access. The backend requires appropriate secret configuration and validation logic. Authorization must also be enforced through suitable checks to ensure that users can access only the operations permitted by the application's access-control rules.

1.4.11 Razorpay Integration

Razorpay is a payment service provider offering payment-processing APIs and related services. TrustFi includes Razorpay-related payment integration in its technology context. Depending on the implemented workflow, a payment integration may support payment initiation and transaction-status handling.

Actual payment functionality depends on the presence of the relevant code, valid credentials, correct configuration, and successful testing. The report should claim successful transaction processing only if the integration has been verified in the current implementation.

1.4.12 AI-Assisted Loan Eligibility and Fraud-Risk Assessment

TrustFi includes eligibility and fraud-risk assessment logic intended to assist the evaluation of submitted loan applications. Such functionality may use application data, configured rules, risk indicators, and document-validation results to support preliminary assessment.

The term AI-assisted should reflect the actual implementation. Rule-based assessment is different from a trained machine-learning model. Unless the source code demonstrates a trained model or an actual AI-service integration, the system should be described as using configured assessment logic rather than claiming a fully trained AI model.

The results of eligibility and fraud-risk assessment are intended to support review and should not be treated as conclusive proof of creditworthiness, fraud, or final loan approval.

1.4.13 Blockchain Technology

Blockchain is a distributed ledger technology in which records are maintained through a network protocol and protected using cryptographic mechanisms. It can be relevant to auditability, integrity, and controlled sharing of information in financial applications.

Blockchain is part of the privacy-aware technology context explored by TrustFi. Its use in the running application must be verified through actual smart contracts, blockchain-network interaction, transaction handling, or related implementation evidence. If these components are not operational, blockchain should be presented as a proposed enhancement rather than a completed system feature.

1.4.14 Fully Homomorphic Encryption (FHE)

Fully Homomorphic Encryption is a cryptographic technique that supports computation on encrypted data under specific mathematical operations. Depending on the scheme and supported operations, results can be obtained without first decrypting the underlying data.

FHE is relevant to privacy-aware financial processing because loan applications may contain sensitive personal and financial information. However, practical use of FHE involves computational overhead, scheme limitations, and careful key management.

In TrustFi, FHE should be documented as an implemented feature only if the project includes an operational encryption library or service and demonstrable encrypted computation. Otherwise, it is an area for future research and development.

1.4.15 Integration of Technologies

The technologies in TrustFi work together through a layered web-application architecture. The frontend provides the user interface, while the backend processes requests and executes the configured application logic. REST APIs connect these layers, and Prisma ORM provides database access. OCR supports document information extraction, JWT supports authentication, and external payment services may support payment-related workflows where configured.

This modular approach helps separate user-interface concerns, backend processing, and data access. It also provides a foundation for improving document verification, assessment logic, application tracking, and privacy-aware processing in future versions of the platform.

2. PROPOSED SYSTEM

TrustFi is proposed as a web-based digital lending platform that provides a structured workflow for loan application submission, document processing, preliminary risk assessment, and administrative review. The system is designed to support property and gold loan applications through a centralized interface for applicants and authorized administrators.

The proposed system combines a Next.js and React frontend with a Node.js and Express.js backend. TypeScript is used for application development, while Prisma ORM provides database access. REST APIs enable communication between the frontend and backend. The application also includes OCR-related functionality for supported documents and JWT-based authentication.

TrustFi aims to reduce repetitive manual work by organizing application information and supporting configured validation and assessment procedures. Its eligibility and fraud-risk functionality is intended to assist the review process rather than independently guarantee approval or establish fraud.

The main components of the proposed system are:

1. **User Management:** Supports user registration, login, and authenticated access according to implemented functionality.
2. **Loan Application Management:** Organizes application details for supported loan categories.
3. **Document Processing:** Provides document upload and OCR-assisted information extraction for supported inputs.
4. **Eligibility Assessment:** Applies configured criteria to support preliminary loan eligibility evaluation.
5. **Fraud-Risk Assessment:** Evaluates available risk indicators through the implemented assessment logic.
6. **Administrative Review:** Provides functionality for authorized personnel to review applications and manage decisions.
7. **Application Tracking:** Supports visibility into application progress through the configured status workflow.
8. **Payment-Related Operations:** Supports payment functionality where the relevant integration is implemented and configured.
9. **Privacy-Aware Design:** Considers the protection of sensitive financial and identity information and provides a basis for exploring advanced privacy-preserving techniques.

The proposed architecture is modular so that individual components can be maintained and enhanced without requiring a complete redesign. The exact behavior of each component depends on the current source code, database configuration, and available integrations.

2.1 FEASIBILITY STUDY

A feasibility study evaluates whether a proposed system can be developed and operated using available technology, resources, and constraints. It considers technical, economic, operational, and schedule-related factors.

For TrustFi, the feasibility study examines the suitability of the selected web technologies, the resources required for development, the practicality of the application workflow, and the time available for completing the academic project.

2.1.1 Technical Feasibility

TrustFi is technically feasible as a web-based application because its core architecture uses established web development technologies. Next.js and React support the frontend, while Node.js and Express.js support backend development. TypeScript helps maintain consistency in application code, and Prisma ORM provides a structured database-access layer.

Tesseract OCR can support text extraction from suitable document inputs, subject to the quality of the documents and the implemented processing pipeline. JWT-based authentication can support protected access when token validation and authorization checks are correctly configured.

The selected architecture supports modular development and testing. However, the reliability of eligibility assessment, fraud-risk evaluation, document verification, and payment processing depends on the actual implementation and appropriate testing. Advanced technologies such as blockchain and Fully Homomorphic Encryption require separate implementation and verification before they can be considered operational components.

Therefore, the project is technically feasible for an academic prototype using the selected technology stack, while production-level lending would require additional security, performance, compliance, and reliability validation.

2.1.2 Economic Feasibility

Economic feasibility considers whether the cost of developing and maintaining the system is reasonable in relation to its intended benefits.

TrustFi uses several widely available development technologies, including React, Next.js, Node.js, Express.js, TypeScript, and Prisma ORM. These technologies have accessible documentation and development tools, which can help reduce initial software acquisition costs.

The project can be developed and tested on a compatible personal computer. However, actual expenses may arise from hosting, domain registration, database services, external APIs, payment processing, and other third-party services, depending on the selected deployment and configuration.

The use of open-source tools can make the academic prototype economical to develop. Production deployment may require additional investment in infrastructure, monitoring, security, backups, and ongoing maintenance.

2.1.3 Operational Feasibility

Operational feasibility evaluates whether the system can be used effectively by its intended users and whether it fits the proposed workflow.

TrustFi is designed to provide applicants with a web interface for entering loan details, submitting supporting documents, and checking application status. Authorized administrators are intended to use the available management interfaces to review applications and record decisions.

The use of a centralized workflow can help organize application information and make the review process easier to follow. OCR-assisted processing and configured assessment logic may reduce repetitive tasks where these features operate correctly.

Operational success depends on an understandable interface, accurate data entry, appropriate access permissions, clear application statuses, and suitable handling of processing errors. Users should also be informed when document extraction or automated assessments require manual verification.

The proposed workflow is operationally feasible for demonstration and evaluation as an academic project. Its suitability for real lending operations would require feedback from intended users and further validation.

2.1.4 Schedule Feasibility

Schedule feasibility determines whether the project can be completed within the time available for the academic submission.

TrustFi is developed using a modular web-application architecture. Its major activities include frontend development, backend API development, database integration, authentication, loan application handling, document processing, assessment logic, and testing.

Dividing these activities into modules helps organize development and makes it possible to test individual components before integrating them. Documentation, diagram preparation, and report validation can be carried out alongside implementation.

The completion of optional or advanced components, including operational blockchain functionality, FHE-based computation, or trained machine-learning models, depends on the available time and the actual project requirements.

Based on the selected technologies and modular approach, the project is feasible as an academic mini project provided that development, integration, testing, and documentation are managed within the available schedule.

2.1.5 Feasibility Study Summary

Feasibility Type	Assessment	Basis
Technical	Feasible for an academic prototype	Established web technologies and a modular architecture
Economic	Generally feasible	Accessible development tools and potential use of open-source technologies
Operational	Feasible with validation	Centralized application workflow and interfaces for applicants and administrators
Schedule	Feasible with prioritization	Modular implementation and phased development

The above assessments are preliminary. They describe the suitability of the proposed approach and do not replace evidence from implementation, testing, or deployment.

2.2 USER REQUIREMENTS

User requirements describe the functions and quality expectations of the system from the perspective of its intended users. TrustFi primarily considers applicants and authorized administrators as its user categories. Additional technical or operational roles may be introduced if supported by the final implementation.

2.2.1 Applicant Requirements

Applicants require an accessible interface through which they can provide the information needed for a loan application.

The main applicant requirements are:

1. The ability to register and log in using the implemented authentication mechanism.
2. The ability to enter personal and contact information.
3. The ability to provide employment, financial, and loan-related details.
4. The ability to select a supported loan category, such as a property loan or gold loan, where available.
5. The ability to upload required identity and supporting documents.
6. The ability to submit application information for processing or review.
7. The ability to receive relevant validation messages when input is incomplete or invalid.
8. The ability to view application status where status tracking is implemented.
9. The ability to access only their permitted application information.
10. The ability to receive clear information when a document requires additional verification.

2.2.2 Administrator Requirements

Administrators require appropriate access to the application-management functionality so they can review submissions and manage application decisions.

The main administrator requirements are:

1. Authenticated access to authorized administrative functions.
2. The ability to view applications permitted by the user's assigned access rights.
3. The ability to inspect submitted application details.
4. The ability to access supporting document information where authorized.
5. The ability to review OCR-extracted information and validation results where available.
6. The ability to examine eligibility-assessment results and configured fraud-risk indicators.
7. The ability to manage application decisions and statuses according to the implemented workflow.
8. The ability to identify incomplete applications or information requiring further review.
9. The ability to avoid unauthorized disclosure of applicant information.
10. The ability to use available administrative functions reliably and consistently.

2.2.3 Functional Requirements

Functional requirements describe the operations that the system is expected to perform.

Requirement ID	Functional Requirement	Description
FR-01	Authentication	The system should support the implemented registration and login workflow.
FR-02	Application Entry	The system should accept relevant loan application information.
FR-03	Loan Categories	The system should support configured property and gold loan workflows.
FR-04	Document Upload	The system should accept supported document inputs.
FR-05	OCR Processing	The system should extract text from supported documents where OCR processing is implemented.
FR-06	Data Validation	The system should perform configured checks on application data and documents.

FR-07	Eligibility Assessment	The system should evaluate application information using implemented eligibility criteria.
FR-08	Fraud-Risk Assessment	The system should evaluate configured risk indicators and make available assessment results.
FR-09	Administrative Review	Authorized users should be able to review applications and manage decisions using implemented functions.
FR-10	Status Tracking	The system should display available application-status information to authorized users.
FR-11	Payment Processing	Payment operations should be available where the payment integration is implemented and configured.
FR-12	Access Control	The system should restrict protected operations according to implemented authorization rules.

These requirements describe the intended system behavior. Each requirement should be mapped to the corresponding implementation and test case before the final report is submitted.

2.2.4 Non-Functional Requirements

Non-functional requirements describe the quality attributes expected of the system.

1. Security: The system should validate authentication tokens, enforce authorization rules, protect sensitive information, and avoid exposing credentials or secrets.

2. Privacy: Applicant identity and financial information should be handled only for authorized purposes and protected against unauthorized access.

3. Usability: The interface should provide understandable labels, clear form instructions, useful validation messages, and consistent navigation.

4. Reliability: The system should handle expected requests and input errors consistently and provide appropriate feedback when an operation fails.

5. Maintainability: The use of modular components and TypeScript should support code organization and future maintenance.

6. Performance: The application should process ordinary requests within acceptable response times under the tested workload. Performance claims should be supported by measurements.

7. Compatibility: The web interface should work with supported browsers and a compatible runtime environment.

8. Data Integrity: Application information and status changes should be validated and stored consistently according to the database design.

9. Scalability: The architecture should allow future improvements in infrastructure and application components as usage increases, subject to suitable engineering and testing.

10. Transparency: Eligibility results and fraud-risk indicators should be presented in a manner that supports review and avoids treating automated outputs as unquestionable decisions.

2.2.5 System Constraints and Assumptions

The system depends on compatible software versions, valid environment configuration, an available database, and correctly configured external services where required. OCR results depend on the quality and format of the submitted documents. Assessment quality depends on the implemented criteria and the accuracy of the input information.

Blockchain, FHE, trained machine-learning models, and payment-processing capabilities should be treated as implemented only when the relevant code, configuration, and test evidence are available. The academic prototype does not, by itself, establish compliance with all financial, privacy, or lending regulations.

These constraints should be considered when evaluating the system and planning future improvements.

3. ANALYSIS AND DESIGN

System analysis and design are important stages in the development of a software application. System analysis identifies the requirements, users, data, and operations of the application, while system design describes the structure and interactions of its components.

For TrustFi, the analysis and design phase focuses on the digital lending workflow, including user authentication, loan application management, document processing, preliminary eligibility assessment, fraud-risk evaluation, and administrative review.

The system is designed around a web-based architecture in which the frontend communicates with the backend through REST APIs. The backend processes requests, applies the configured business logic, and accesses application data through Prisma ORM. Document-processing and assessment components support the lending workflow where implemented.

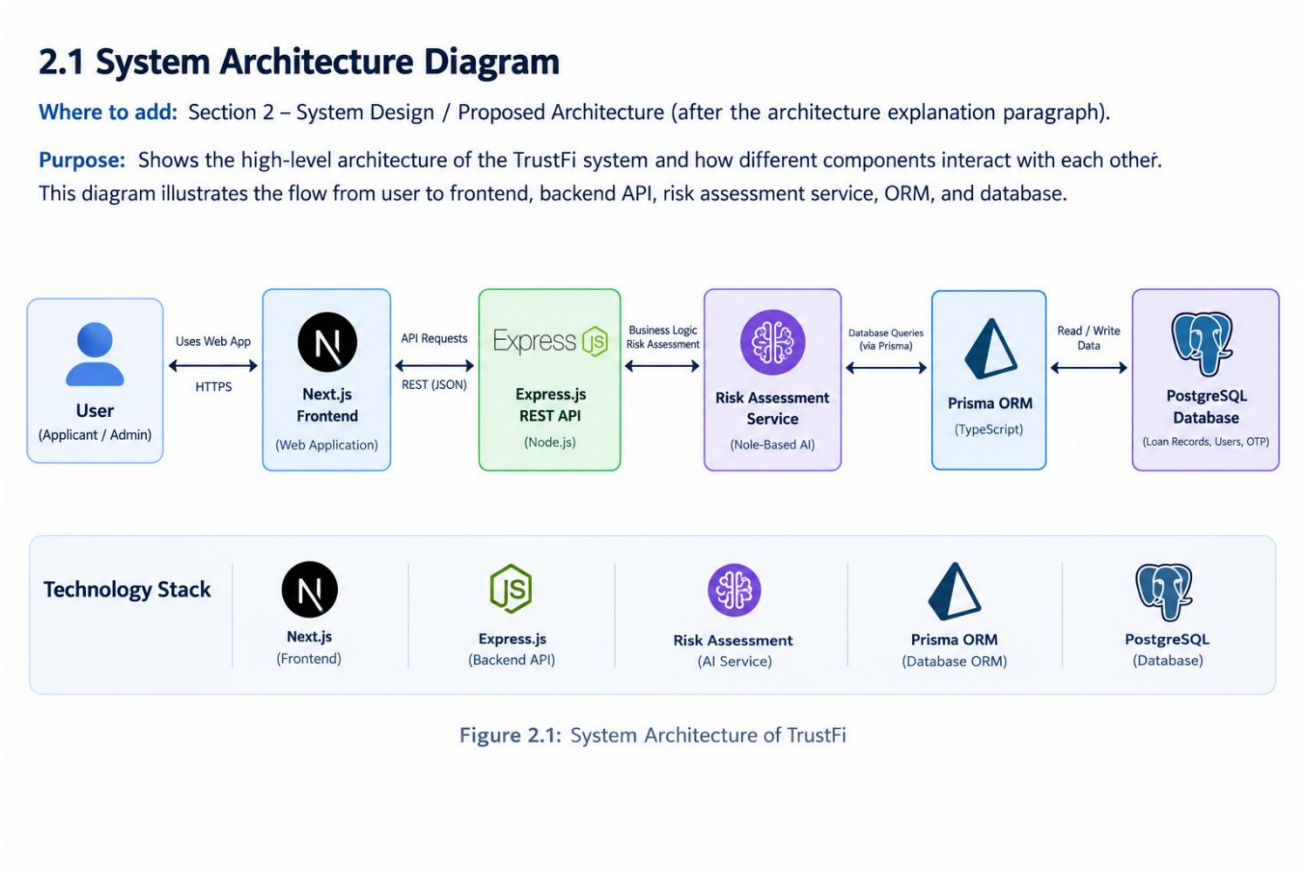


Figure 2.1: System Architecture of TrustFi

The following diagrams represent the conceptual design of TrustFi. They should be checked against the actual source code and database configuration before final submission.

3.1 OBJECT DIAGRAM

An object diagram is a UML (Unified Modeling Language) diagram that represents a snapshot of a system at a particular point in time. It shows example objects, their attribute values, and the relationships between them.

In TrustFi, an object diagram can illustrate how an applicant, a loan application, supporting documents, an assessment result, and an administrator may be associated during the application-review process.

Description of the Object Diagram

The proposed object diagram contains the following representative objects:

1. **Applicant Object:** Represents a user who submits a loan application.
2. **Loan Application Object:** Represents an application containing the relevant loan category and application details.
3. **Document Object:** Represents a supporting document submitted for the application.
4. **Assessment Result Object:** Represents the output of configured eligibility or fraud-risk assessment logic.
5. **Administrator Object:** Represents an authorized user who reviews the application and manages its status.

These are conceptual UML objects for illustrating the workflow. Their names and attributes are examples and must not be interpreted as confirmed database tables or classes unless they match the actual implementation.

Object Diagram — TrustFi

Figure 3.1: Conceptual Object Diagram of TrustFi

Explanation

The diagram illustrates a representative loan application workflow. An applicant submits a loan application and provides the required supporting documents. The application may be associated with an assessment result generated by the configured assessment logic. An authorized administrator reviews the application and manages its status according to the available workflow.

The diagram represents a conceptual snapshot rather than a verified runtime instance. The example status and object attributes are illustrative only. Actual object structures, attribute names, relationships, and status values must be confirmed from the project's implementation.

3.2 CLASS DIAGRAM

A class diagram is a UML structural diagram that represents the classes of a system, their attributes, operations, and relationships. It provides a static view of the system and helps explain how different components may be organized.

In TrustFi, the class diagram is intended to represent the main entities involved in the digital lending workflow, including users, loan applications, supporting documents, assessment results, and administrative review.

Description of the Class Diagram

The conceptual class structure of TrustFi includes the following components:

1. User Class

The User class represents a person who accesses the platform. Depending on the implemented role-management mechanism, a user may act as an applicant or an administrator. The class may include identity and account-related information and operations associated with authentication and profile management.

2. LoanApplication Class

The LoanApplication class represents a loan request submitted through the platform. It may contain application details, the loan category, relevant financial information, and the current application status. Its operations may include application submission and status management.

3. Document Class

The Document class represents supporting documents uploaded as part of a loan application. It may include document metadata, file references, processing status, and extracted information where available. Document handling must follow the application's implemented validation and access-control rules.

4. AssessmentResult Class

The AssessmentResult class represents the output of the configured eligibility and fraud-risk assessment processes. It may contain assessment values, risk indicators, and related evaluation information. The actual attributes depend on the implemented assessment logic.

5. Administrative Review

Administrative review represents the process through which authorized personnel examine applications and manage decisions. This responsibility may be implemented through methods associated with a user role, a separate class, or backend service functions. The final class diagram must reflect the actual source code.

Relationships Between Classes

The conceptual relationships are described below:

- A user can submit one or more loan applications, subject to the application's rules.
- A loan application can be associated with supporting documents.
- A loan application may have eligibility and fraud-risk assessment information.
- An authorized administrator can review an application and manage its status.
- Authentication and authorization control access to protected operations.

These relationships describe the proposed logical model. They do not confirm the exact database relationships or class definitions used in the current implementation.

Importance of the Class Diagram

The class diagram helps organize the system's responsibilities and provides a basis for understanding the relationships between application data and business operations. It can assist in planning database models, backend services, and future maintenance.

For the final report, the class names, attributes, methods, and relationships should be aligned with the actual TrustFi source code and Prisma schema. Conceptual classes should not be presented as implemented classes unless they exist in the application.

3.3 USE CASE DIAGRAM

A use case diagram is a UML behavioral diagram that represents the interactions between users (actors) and the functions provided by a system. It defines the system's functional boundaries and illustrates how different users interact with its services.

In TrustFi, the use case diagram describes the major activities associated with digital loan application management. The primary actors are the Applicant and the Administrator. External services may also participate in specific workflows where the corresponding integrations are implemented.

Actors of the System

1. Applicant

The Applicant is the primary user who accesses the platform to submit and track loan applications. The applicant interacts with the available features through the web interface.

The applicant's intended activities include:

- Registering and logging in to the platform.
- Entering personal, financial, and loan-related information.
- Selecting a supported loan category.
- Uploading supporting documents.
- Submitting a loan application.
- Viewing available eligibility or risk-assessment information where exposed by the application.
- Tracking the application status.
- Accessing payment-related functionality where implemented and applicable.

2. Administrator

The Administrator is an authorized user responsible for reviewing and managing loan applications through the available administrative functionality.

The administrator's intended activities include:

- Logging in to the platform.

- Viewing applications permitted by assigned access rights.
- Reviewing application details and submitted documents.
- Examining available document-processing and assessment results.
- Managing application decisions and status updates according to the configured workflow.

3. External Payment Service

A payment service, such as Razorpay, may participate in payment-related use cases if the integration is operational. Its responsibilities may include processing a payment request and returning transaction-related information. The exact interaction depends on the implemented integration.

Main Use Cases

The major use cases of TrustFi are described below:

User Authentication: Enables users to access the platform through the configured authentication mechanism.

Loan Application Submission: Allows applicants to provide the required information and submit a loan request.

Document Upload and Processing: Supports uploading documents and extracting text from supported inputs through OCR functionality where implemented.

Eligibility Assessment: Evaluates application information against configured eligibility criteria.

Fraud-Risk Assessment: Evaluates available fraud-risk indicators through the implemented assessment logic.

Application Review: Allows authorized administrators to inspect submitted applications and available supporting information.

Application Status Management: Supports updates to application status according to the configured workflow.

Application Tracking: Allows applicants to view the status information available for their applications.

Payment Processing: Supports payment-related activities where the relevant integration has been implemented and configured.

System Boundary and Access Control

The system boundary includes the TrustFi web interface and the backend functionality provided by its APIs. External services are treated as separate systems when they participate in the application workflow.

Access to protected use cases must be controlled through authentication and authorization checks. Applicants should be restricted to permitted operations and application information, while administrative operations should be available only to appropriately authorized users.

The use case diagram provides a high-level view of the intended interactions. Before final submission, the actors and use cases should be compared with the actual frontend routes, backend API endpoints, access-control logic, and available integrations.

3.4 SEQUENCE DIAGRAM

A sequence diagram is a UML interaction diagram that illustrates the sequence of messages exchanged between system components over time. It represents the order in which operations occur and explains how different participants collaborate to complete a particular process.

In TrustFi, sequence diagrams can be used to represent interactions between the applicant, frontend interface, backend API, database layer, and document-processing or assessment components. These interactions help explain the flow of information during loan application submission and review.

Participants in the Sequence Diagram

1. Applicant: The user who enters loan details, uploads supporting documents, and submits an application.

2. Frontend Interface: The Next.js and React interface through which the applicant interacts with the platform.

3. Backend API: The Express.js backend that receives requests, validates input, and executes the configured business logic.

4. Database Layer: The data-access layer that uses Prisma ORM to perform database operations against the configured database.

5. Document Processing Component: The component responsible for OCR-related processing of supported documents where implemented.

6. Assessment Component: The logic responsible for evaluating application information using the configured eligibility and fraud-risk criteria.

7. Administrator: The authorized user who reviews submitted applications and manages decisions according to the available workflow.

Sequence of Loan Application Processing

The proposed loan application sequence consists of the following steps:

1. The applicant opens the TrustFi web application and accesses the loan application interface.
2. The applicant enters the required personal, financial, and loan-related details.
3. The applicant uploads the required supporting documents.
4. The frontend sends the application information to the backend through the relevant API request.
5. The backend validates the incoming request and performs the configured checks.

6. The backend uses the database-access layer to store or retrieve application information as required by the implemented workflow.
7. Where OCR processing is configured, the supported document input is passed to the document-processing component for text extraction.
8. Extracted information and application details are processed by the available validation and assessment logic.
9. The backend returns the appropriate response to the frontend, including confirmation or validation feedback according to the operation's outcome.
10. The applicant views the response and available application-status information.
11. An authorized administrator reviews the application and available assessment information.
12. The administrator manages the application decision or status through the available administrative functionality.

The exact ordering of document storage, OCR execution, assessment, and application-status updates depends on the implemented backend workflow. The sequence diagram prepared for the final report should therefore be verified against the actual API endpoints and service calls.

Importance of the Sequence Diagram

The sequence diagram helps explain the communication between the frontend, backend, database-access layer, and supporting components. It makes the order of operations easier to understand and helps identify where validation, document processing, assessment, and error handling occur.

It can also support implementation testing by identifying expected interactions and responses at different stages of the application workflow. For TrustFi, the sequence diagram provides a conceptual view of loan application processing and administrative review without assuming that every proposed integration is operational.

3.5 ACTIVITY DIAGRAM

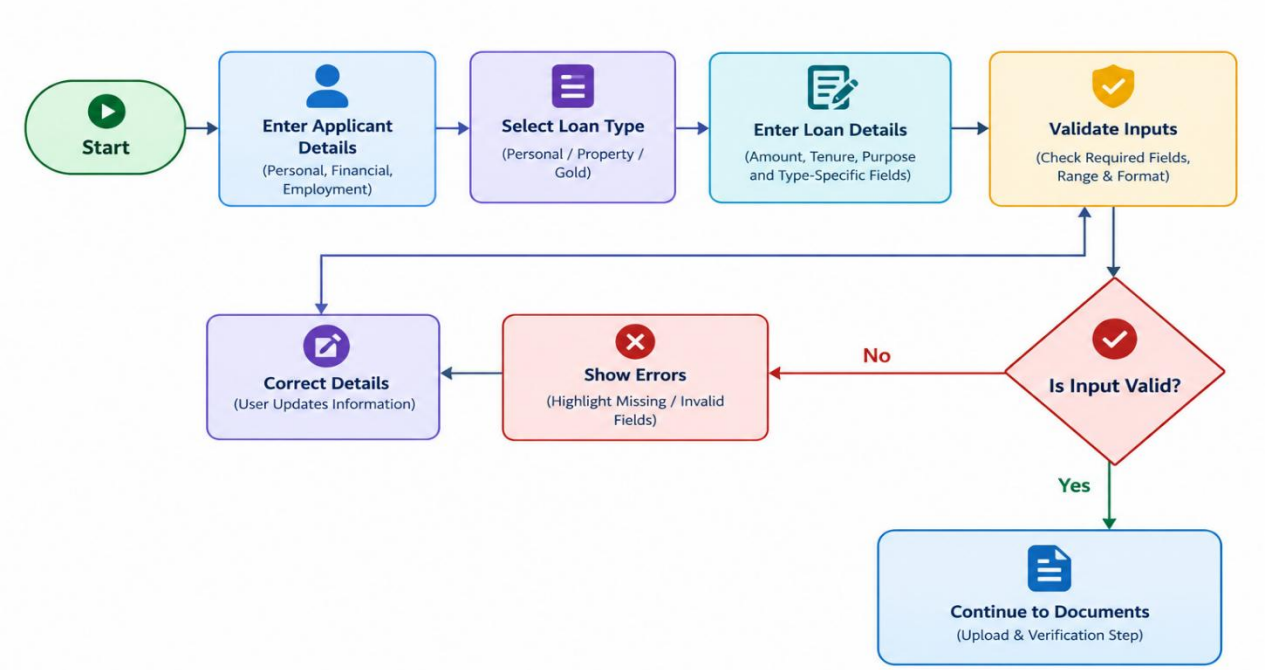


Figure 3.1: Loan Application Workflow

An activity diagram is a UML behavioral diagram that represents the sequence of activities performed within a system. It illustrates the workflow from the initial activity to the final outcome and can include decisions, alternative paths, and parallel activities.

In TrustFi, the activity diagram describes the major steps involved in processing a loan application, from user authentication and information entry to document processing, assessment, and administrative review.

Description of the Activity Diagram

The loan application workflow consists of the following activities:

1. Start and User Authentication

The workflow begins when the applicant accesses the TrustFi platform. The applicant proceeds through the configured login or authentication process. If authentication fails, the system provides appropriate feedback and the user must follow the available authentication procedure.

2. Loan Application Entry

After accessing the platform, the applicant enters the required personal, financial, employment, and loan-related information. The applicant selects a supported loan category, such as a property loan or gold loan, where available.

3. Input Validation

The system checks the submitted information against the validation rules implemented in the application. If required information is missing or invalid, the user receives feedback and can correct the input before proceeding.

4. Document Upload

The applicant uploads the supporting documents required for the application. The system processes the uploaded files according to its supported formats and implemented validation procedures.

5. OCR-Based Information Extraction

Where OCR processing is implemented and applicable, the system uses Tesseract OCR to extract text from supported document inputs. Extracted information may require additional validation because OCR results can contain errors.

6. Eligibility and Fraud-Risk Assessment

The application information and available extracted data are processed using the configured eligibility and fraud-risk assessment logic. The results support preliminary evaluation and may identify information requiring additional review.

7. Application Submission and Review

The application proceeds through the configured submission workflow. An authorized administrator reviews the available application information, supporting documents, and assessment results.

8. Decision and Status Update

The administrator manages the application decision or status according to the available workflow and applicable review procedures. The applicant can view the resulting status information where tracking functionality is implemented.

9. Completion

The workflow reaches its end when the relevant processing and review activities are completed for that stage. Additional verification or processing may be required depending on the application outcome and the configured lending workflow.

Decision Points in the Workflow

The main decision points represented in the activity diagram are:

- **Authentication Check:** Determines whether the user has successfully authenticated.
- **Input Validation:** Determines whether the submitted application information satisfies the implemented validation rules.
- **Document Processing:** Determines whether the uploaded document can be processed through the supported workflow.
- **Assessment Review:** Identifies whether the available assessment information requires further examination.
- **Administrative Decision:** Represents the review outcome recorded by an authorized administrator.

These decision points describe the intended workflow. Their exact conditions and possible outcomes must match the implemented application logic.

Importance of the Activity Diagram

The activity diagram provides a clear representation of the overall loan application workflow. It helps explain how applicants and administrators interact with the platform, where validation occurs, and how application information moves through processing and review.

It also helps identify possible error paths, incomplete submissions, and stages requiring manual intervention. This supports workflow analysis, implementation planning, and the preparation of relevant functional test cases.

The activity diagram for TrustFi should reflect the actual order of operations and should not imply automatic loan approval, definitive fraud detection, or successful payment processing unless those outcomes are supported by the implementation.

3.6 MODULE HIERARCHY DIAGRAM

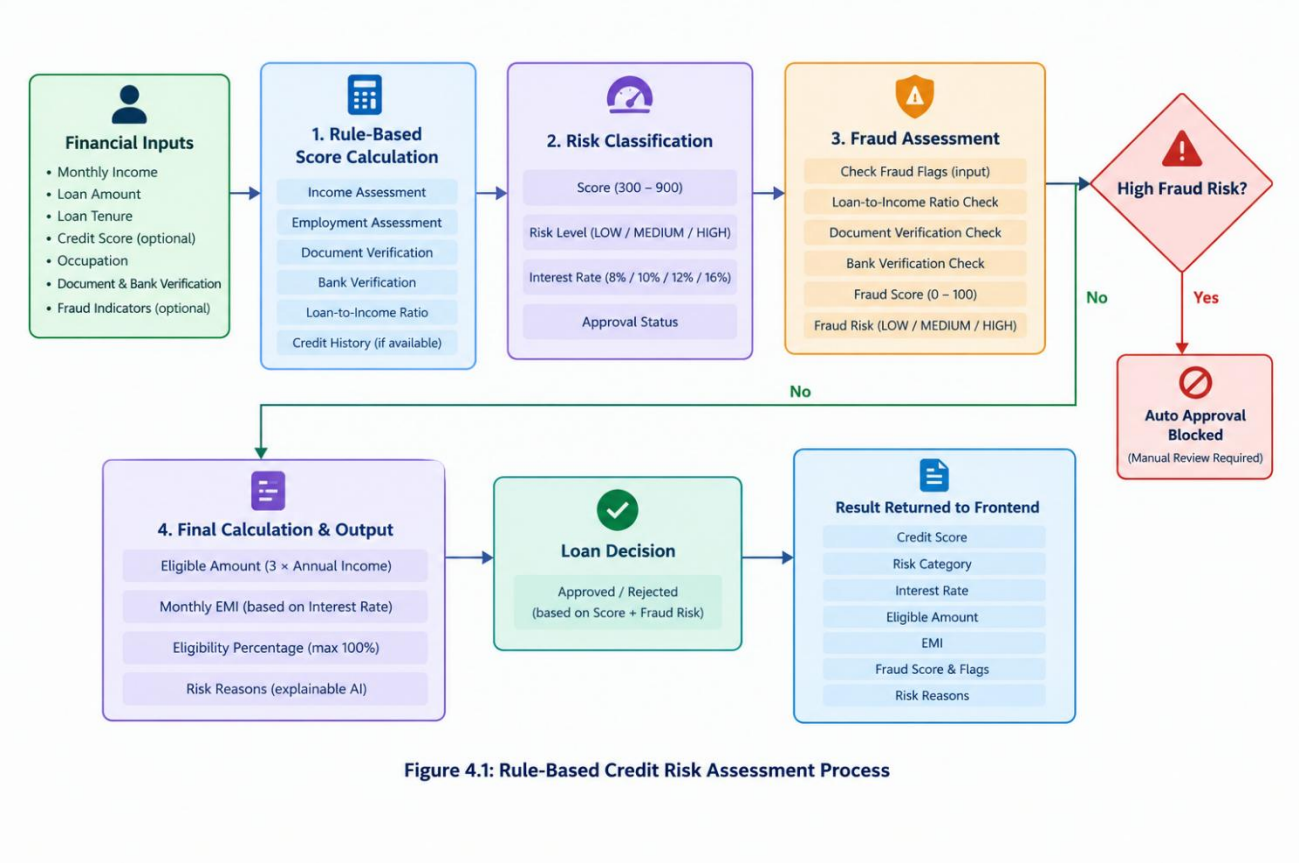


Figure 4.1: Rule-Based Credit Risk Assessment Process

A module hierarchy diagram represents the hierarchical organization of a software system by dividing it into major modules and their related submodules. It provides a high-level view of the system's functional structure and helps explain how different responsibilities are organized.

TrustFi is designed as a modular digital lending platform. Its major functional areas include user authentication, loan application management, document processing, eligibility assessment, fraud-risk evaluation, application tracking, and administrative review. These modules work together through the frontend, backend APIs, and database-access layer.

Description of the Module Hierarchy

The conceptual module hierarchy of TrustFi consists of the following modules:

1. User Authentication and Authorization Module

This module handles the authentication-related workflow and supports controlled access to protected application functions. JWT-related functionality is used for authentication, while authorization checks determine whether a user is permitted to perform a particular operation.

2. Loan Application Management Module

This module manages the information associated with loan applications. It supports the application workflow for property and gold loans, according to the categories and operations available in the implementation.

3. Document Management Module

This module supports uploading and handling supporting documents. It provides the document inputs required for processing and review, subject to the application's file-handling and validation rules.

4. OCR and Information Extraction Module

This module uses Tesseract OCR for supported document-processing tasks where implemented. It assists in extracting text from document inputs so that the resulting information can be examined by validation or assessment logic.

5. Loan Eligibility Assessment Module

This module evaluates application information against the configured eligibility criteria. Its purpose is to support preliminary assessment rather than guarantee loan approval.

6. Fraud-Risk Assessment Module

This module evaluates available fraud-risk indicators using the implemented assessment logic. The results can support further examination of applications, but they do not independently prove fraudulent activity.

7. Application Tracking Module

This module provides access to available application-status information. It helps applicants understand the progress of their applications according to the configured status workflow.

8. Administrative Review Module

This module supports authorized administrative activities, including reviewing application information, examining available assessment results, and managing application decisions or statuses.

9. Payment Integration Module

This module represents payment-related functionality associated with the Razorpay integration. Its operational scope depends on the implementation, configuration, and testing of the relevant payment workflow.

10. Database Access Module

This module represents the data-access responsibilities handled through Prisma ORM and the configured database provider. It supports database operations required by the application's backend services.

Module Interaction

The frontend provides access to the application's user-facing functions and communicates with backend APIs. The backend coordinates request validation, business logic, authentication-related checks, and database operations. Document processing and assessment logic support the relevant loan application workflows.

The modules are conceptually separated according to their responsibilities. However, this hierarchy does not imply that each module exists as an independent code directory, service, or deployable component. The final diagram should be aligned with the actual project structure and implemented functionality.

Advantages of Modular Design

The modular organization provides several advantages:

1. **Maintainability:** Related functionality can be examined and updated more easily.
2. **Code Organization:** Different application responsibilities can be separated logically.
3. **Testing:** Individual functions and workflows can be tested independently where practical.
4. **Extensibility:** Additional features can be introduced without redesigning the entire application.
5. **Readability:** The overall application structure becomes easier for developers and evaluators to understand.
6. **Integration:** Clearly defined responsibilities help organize communication between the frontend, backend, database layer, and supporting components.

The module hierarchy provides a conceptual overview of TrustFi's functional organization. Before final submission, it should be compared with the actual source-code directories, backend routes, service functions, and integrations.

3.8 TABLE SPECIFICATIONS

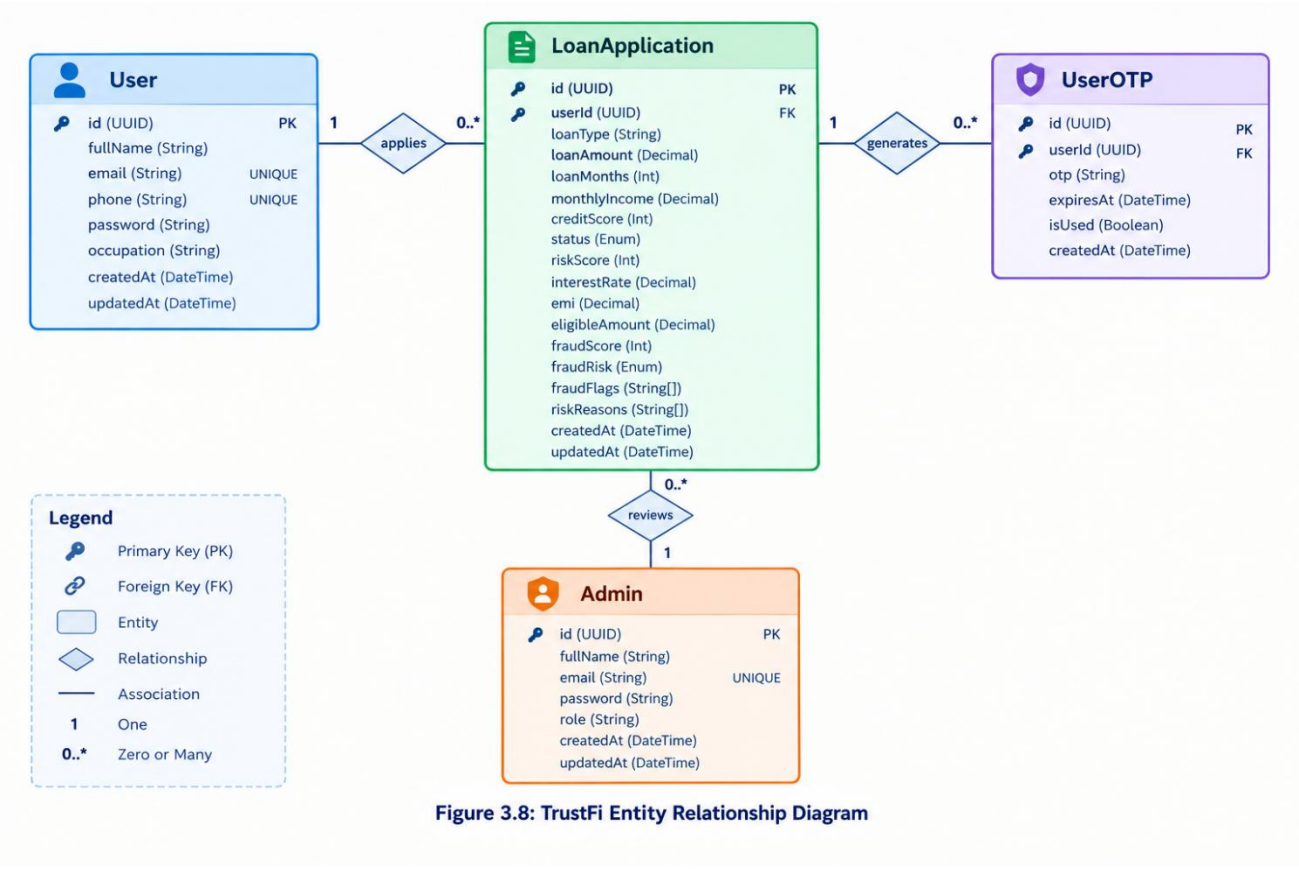


Figure 3.8: TrustFi Entity Relationship Diagram

The database design of TrustFi is intended to organize and manage information related to users, loan applications, uploaded documents, eligibility assessments, and application reviews. The application uses Prisma ORM to interact with its configured database. The exact database structure depends on the models defined in the project's Prisma schema.

The following are the conceptual data entities relevant to the TrustFi platform. These descriptions are intended to explain the information managed by the system and must be aligned with the actual implemented schema before final submission.

3.8.1 User Information

This entity stores information required to identify users and manage their access to the platform. Depending on the implemented model, it may contain a unique identifier, name, email address, authentication-related information, role, and account timestamps. Passwords or authentication secrets should not be stored as plain text.

3.8.2 Loan Application Information

This entity represents loan requests submitted through TrustFi. Relevant information may include the application identifier, applicant reference, loan category, requested amount, application status, submission date, and other details required for assessment.

3.8.3 Document Information

This entity represents documents uploaded in support of a loan application. Depending on the actual implementation, it may record the document type, associated application, storage reference, upload

timestamp, and OCR processing status. Sensitive document contents should be accessible only to authorized users and handled according to the application's security requirements.

3.8.4 Eligibility and Risk Assessment Information

This entity represents the outcome of the eligibility or risk-assessment process, if assessment results are persisted by the application. Relevant fields may include the associated application, assessment outcome, calculated indicators, risk category, and assessment timestamp. The report must distinguish rule-based calculations from any genuinely implemented machine-learning model.

3.8.5 Application Review Information

This entity represents administrative review activity, where supported by the implementation. It may include the application reference, reviewer reference, review decision, remarks, and review timestamp.

3.8.6 Relationships Between Data Entities

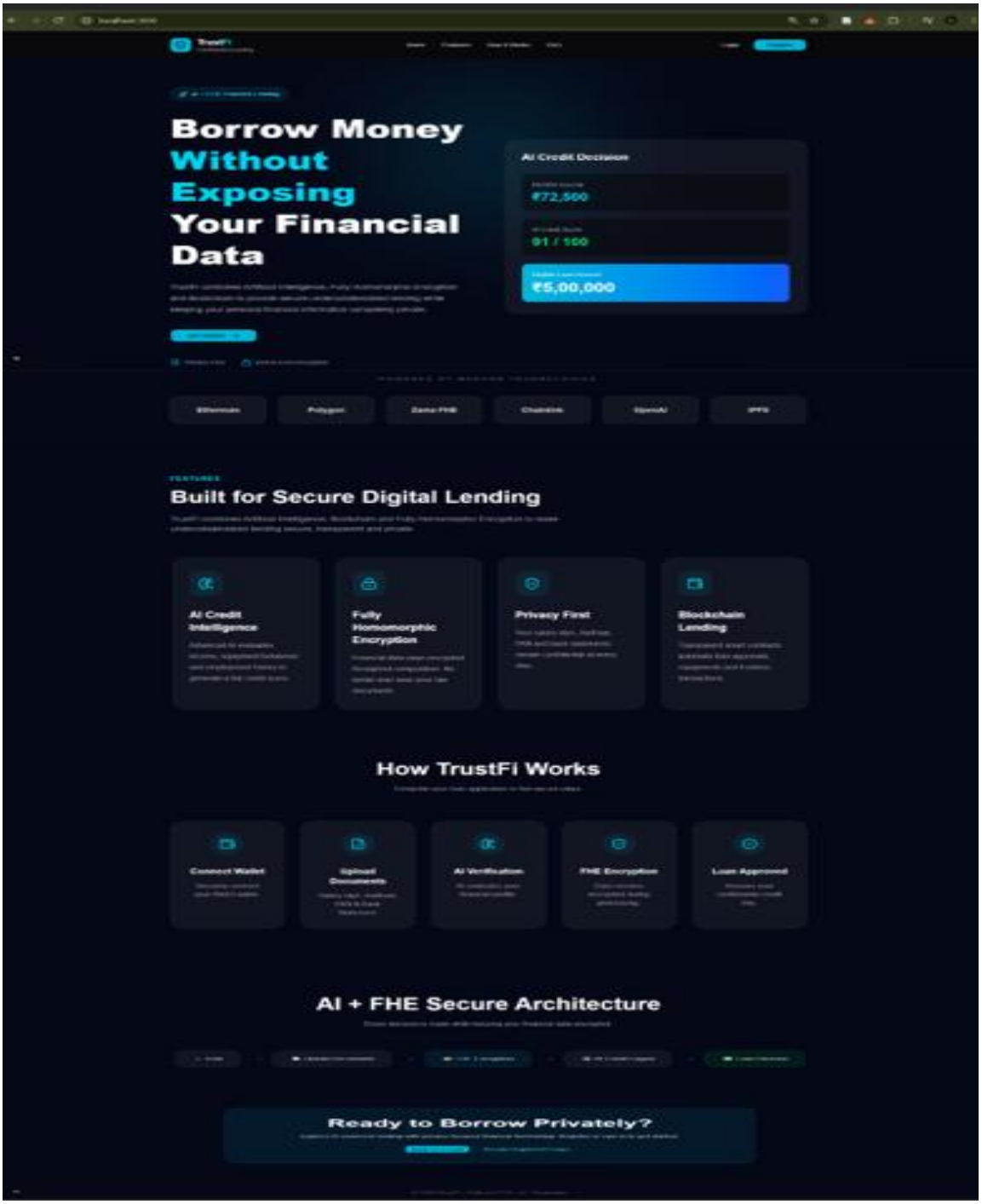
The conceptual design associates users with their loan appli cations and applications with their supporting documents and assessment records. Administrative review information may also be associated with the relevant application and reviewer. The actual relationships, cardinalities, constraints, and field names must be taken from the project's Prisma schema rather than assumed.

3.8.7 Database Integrity and Security

Unique identifiers help distinguish individual records, while appropriate relationships and constraints help maintain data consistency. Authentication, authorization, input validation, and controlled access to sensitive information are important for protecting applicant data. Database indexes, deletion rules, and other constraints should be documented only where they are present in the actual implementation.

4. INPUT AND OUTPUT SCREENS

4.1 INTRODUCTION



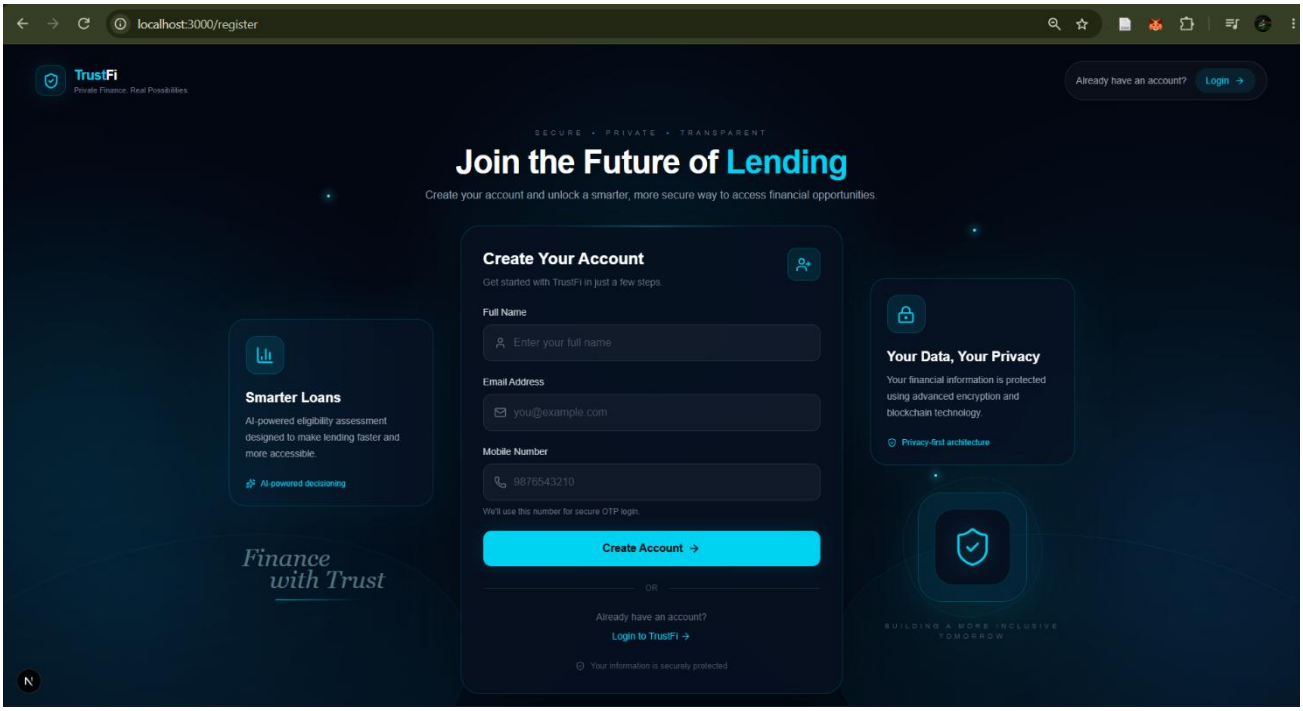
The Input and Output Screens section describes how users interact with the TrustFi platform. The user interface is designed to collect the information required for loan applications, support document submission, display application progress, and present relevant results to users and administrators.

Input screens accept information from users, while output screens display application details, processing results, status updates, and administrative review information. The availability and appearance of each screen depend on the implemented features of the application.

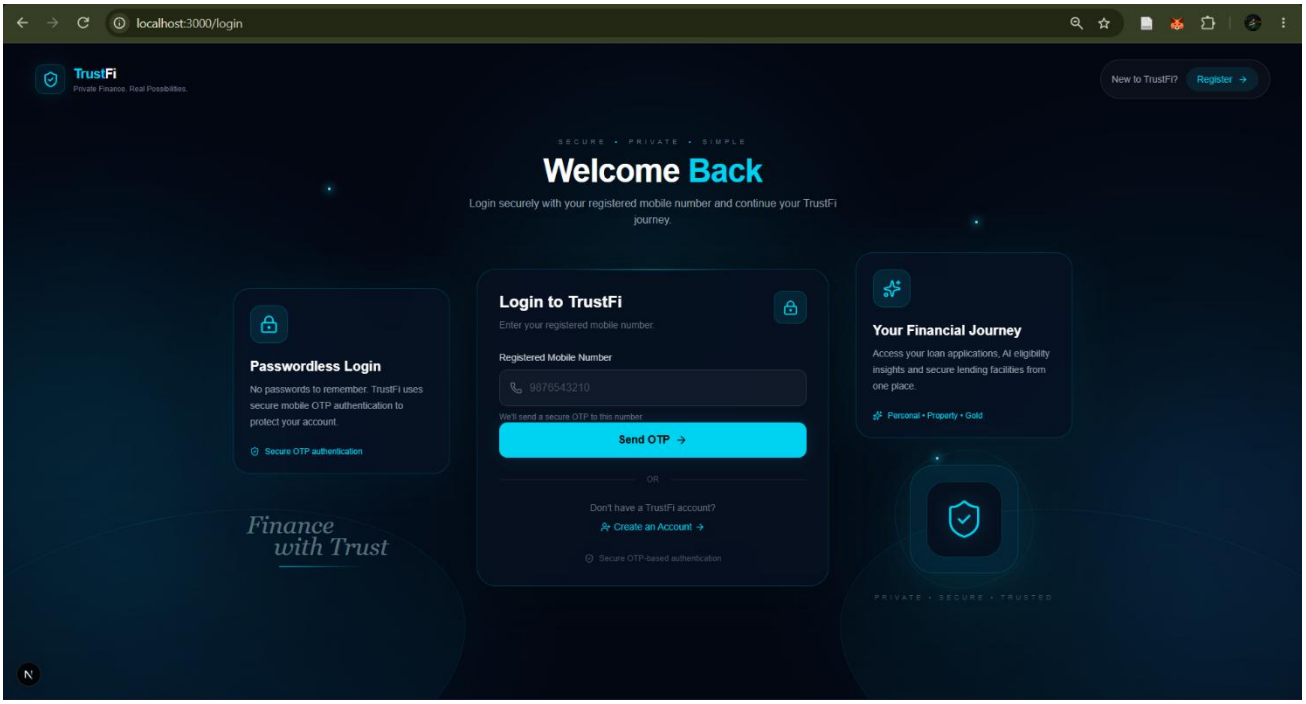
4.2 INPUT SCREENS

4.2.1 User Authentication Screen

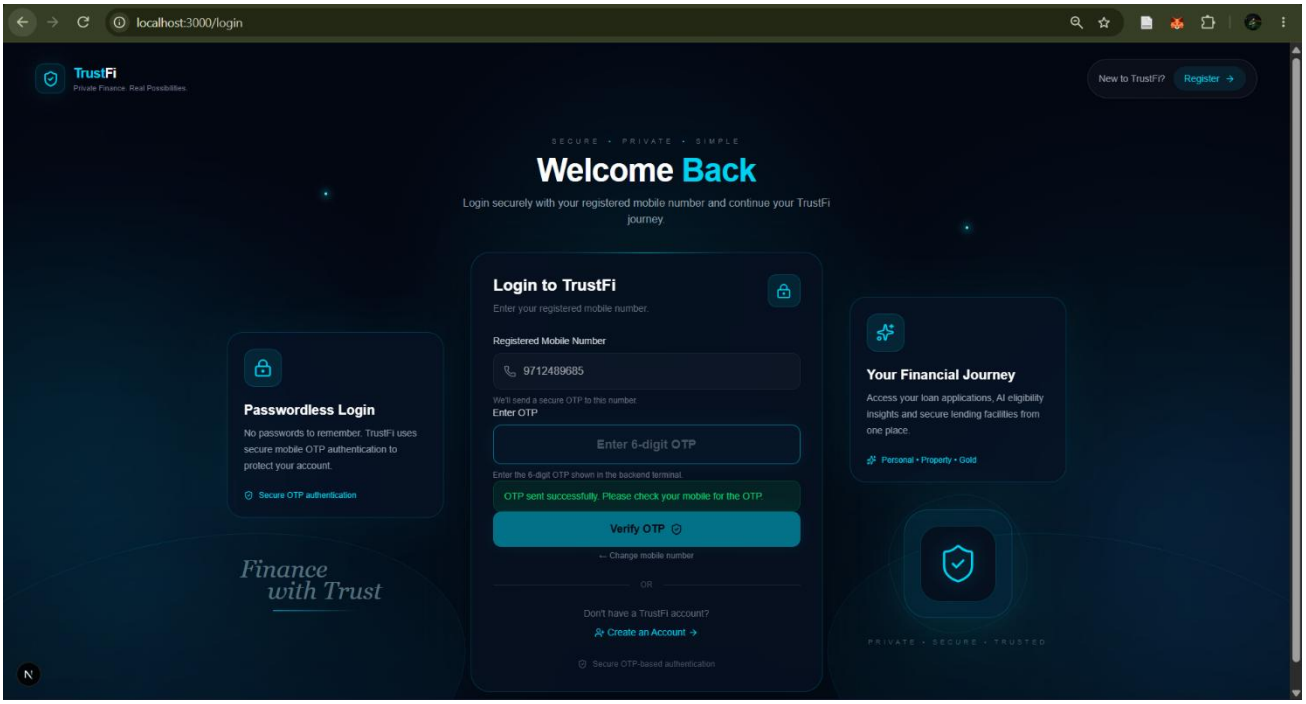
→ Register Page



→Login Page

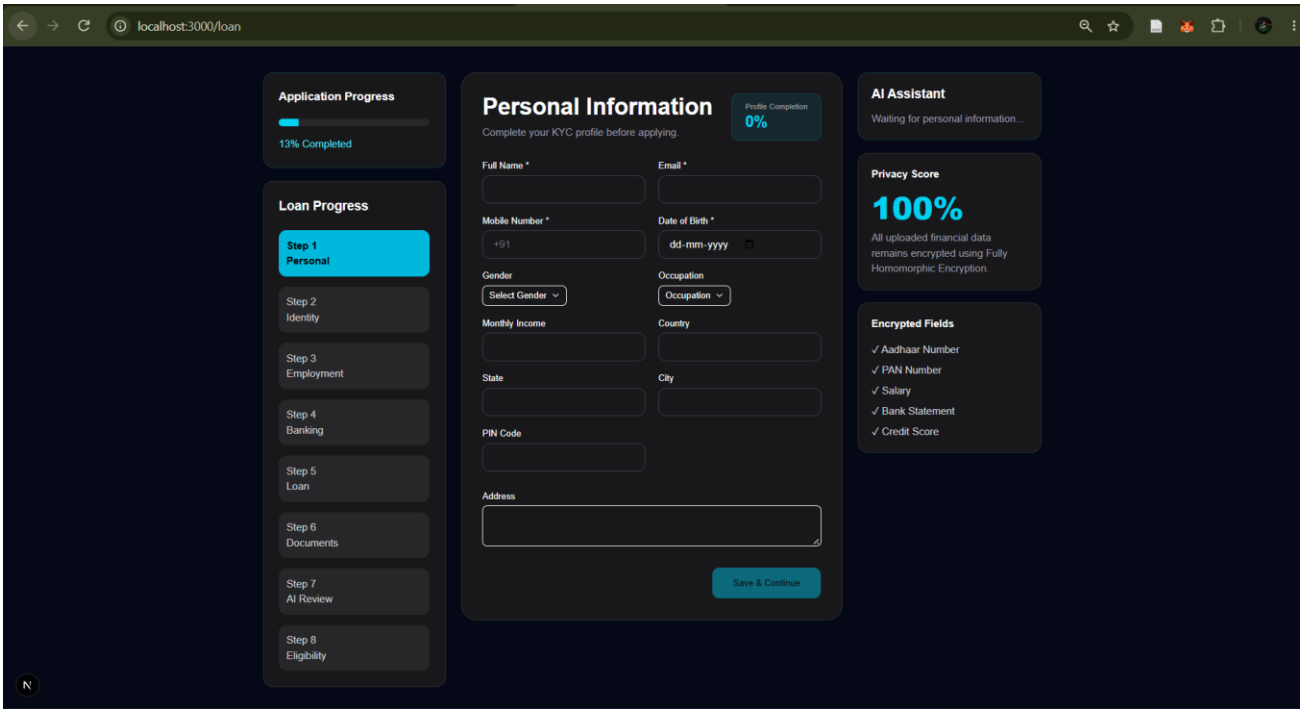


→OTP Verification



The authentication interface allows registered users to access the platform using their credentials. Authentication helps ensure that access to application features is restricted according to the user's permissions.

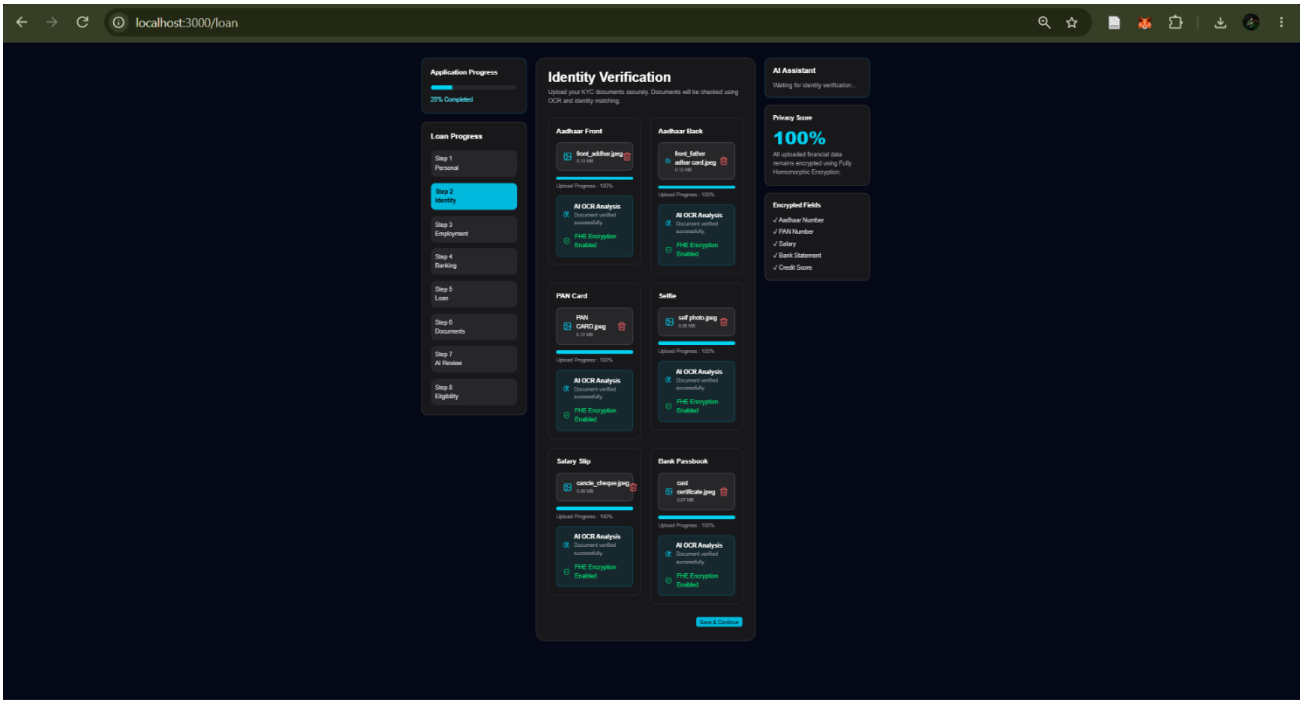
4.2.2 Loan Application Screen



The loan application interface collects the information required to initiate a loan request. Depending on the selected loan category, the form may collect applicant details, requested loan amount, loan purpose, and relevant property or gold-related information.

The system should validate required fields before accepting the application to reduce incomplete or invalid submissions.

4.2.3 Document Upload Screen

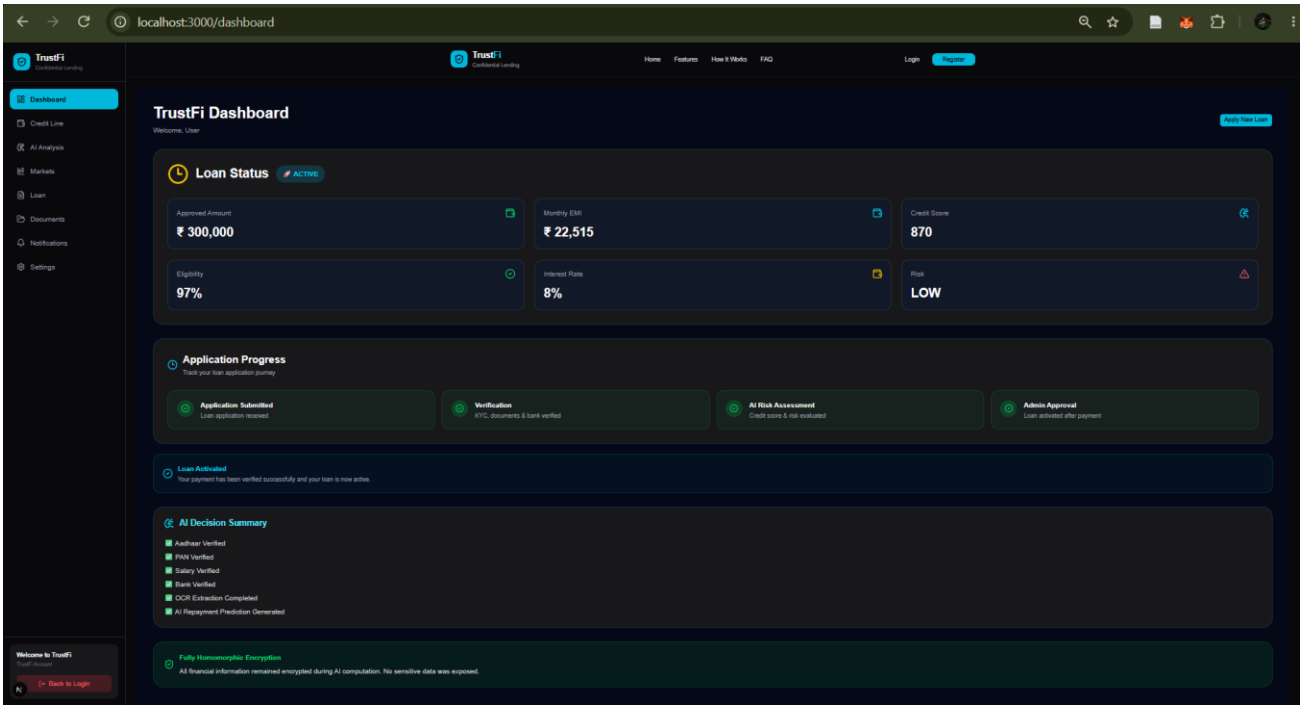


The document upload interface enables applicants to submit supporting documents required for the loan assessment process. Uploaded documents may be processed using Optical Character Recognition (OCR), where implemented, to extract relevant text for further processing.

The interface should communicate upload status and validation errors clearly.

4.3 OUTPUT SCREENS

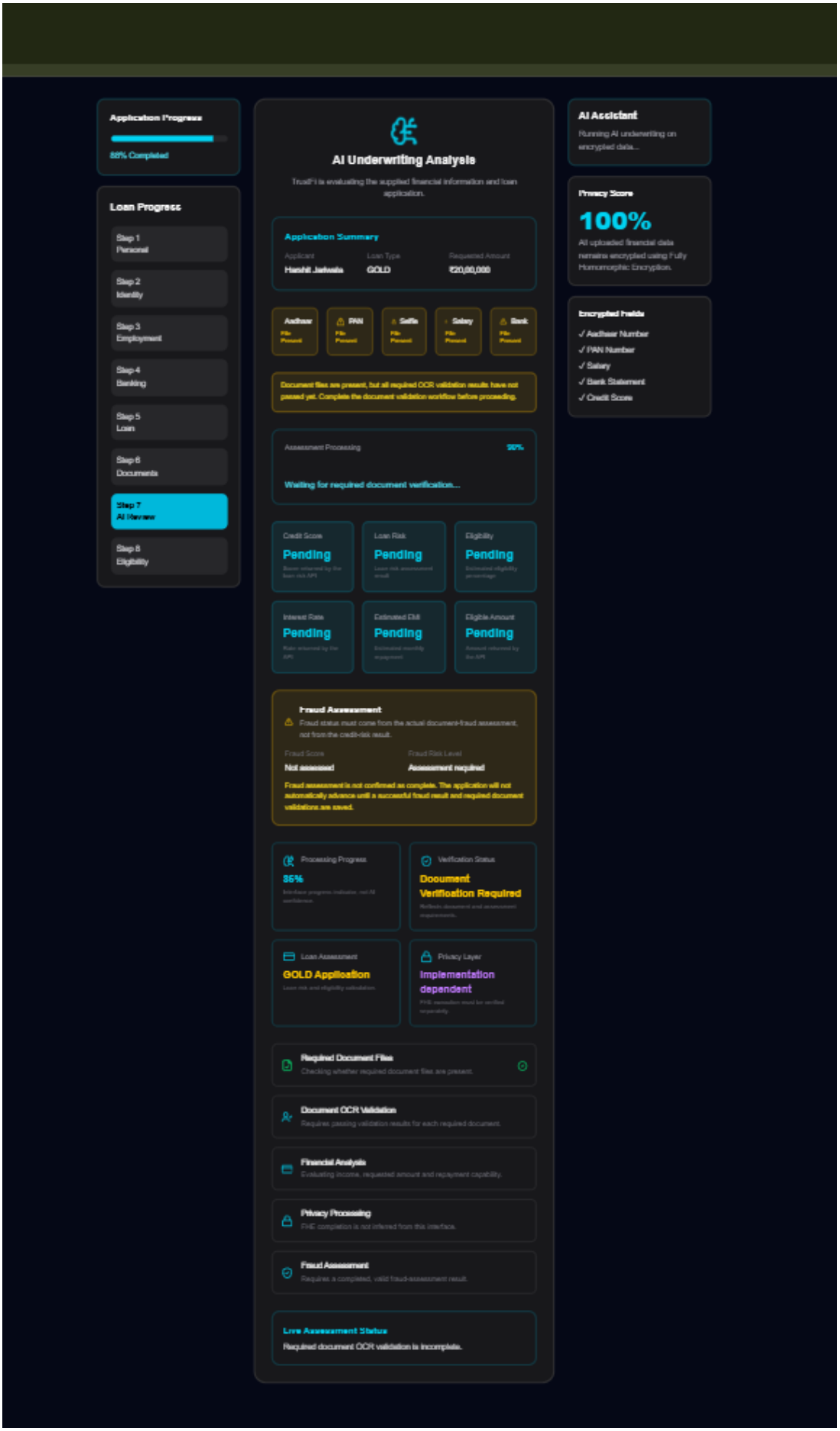
4.3.1 User Dashboard



The dashboard provides an overview of the information available to the authenticated user. Depending on the implemented features, it may display application summaries, recent activity, and links to relevant platform functions.

Screenshot to be added: Actual user dashboard.

4.3.2 Application Status Screen

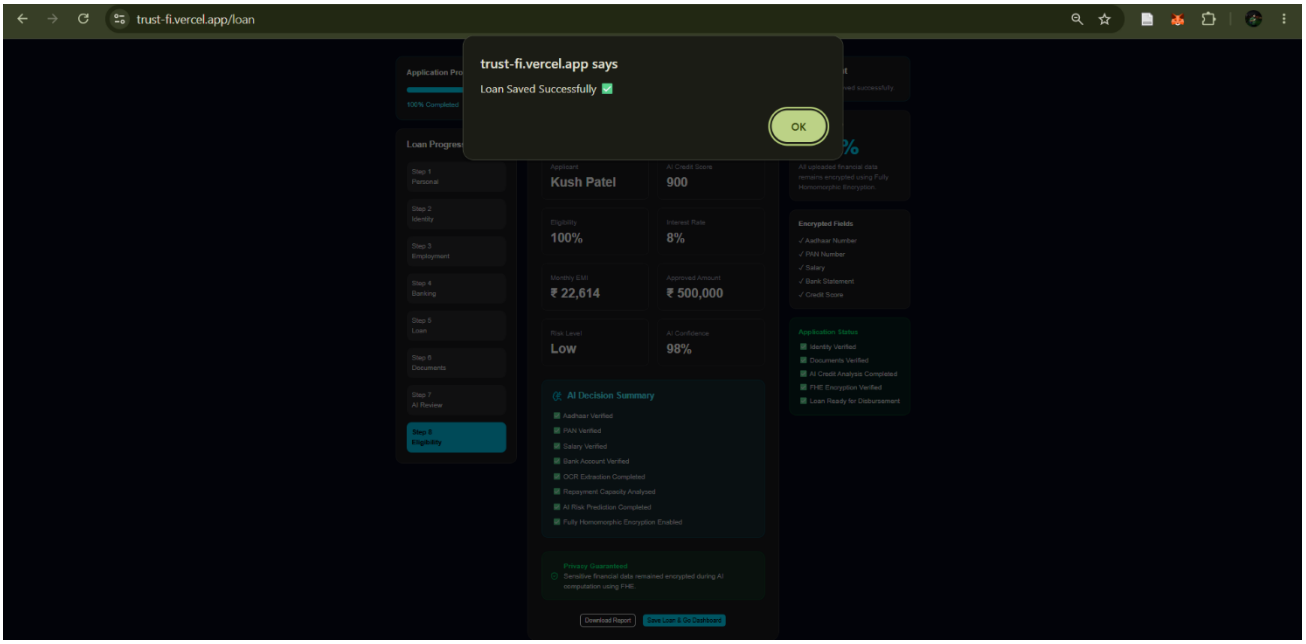

Application Progress
88% Completed
Loan Progress
Step 1
Personal
Step 2
Identity
Step 3
Employment
Step 4
Banking
Step 5
Loan
Step 6
Documents
Step 7
AI Review
Step 8
Eligibility
AI Underwriting Analysis
TrustFi is evaluating the supplied financial information and loan application.
Application Summary
Applicant
Harshit Jaiswala
Loan Type
GOLD
Requested Amount
₹20,00,000
Aadhaar
File Present
PAN
File Present
Selfie
File Present
Salary
File Present
Bank
File Present
Document files are present, but all required OCR validation results have not passed yet. Complete the document validation workflow before proceeding.
Assessment Processing
95%
Waiting for required document verification...
Credit Score
Pending
Loan Risk
Pending
Eligibility
Pending
Interest Rate
Pending
Estimated EMI
Pending
Eligible Amount
Pending
Fraud Assessment
Fraud status must come from the actual document-fraud assessment, not from the credit-risk result.
Fraud Score
Not assessed
Fraud Risk Level
Assessment required
Fraud assessment is not confirmed as complete. The application will not automatically advance until a successful fraud result and required document validations are saved.
Processing Progress
95%
Verification Status
Document Verification Required
Loan Assessment
GOLD Application
Privacy Layer
Implementation dependent
Required Document Files
Checking whether required document files are present.
Document OCR Validation
Requires passing validation results for each required document.
Financial Analysis
Evaluating income, requested amount and repayment capability.
Privacy Processing
FHE completion is not inferred from this interface.
Fraud Assessment
Requires a completed, valid fraud-assessment result.
Live Assessment Status
Required document OCR validation is incomplete.
AI Assistant
Running AI underwriting on encrypted data...
Privacy Score
100%
All uploaded financial data remains encrypted using Fully Homomorphic Encryption.
Encrypted Fields
✓ Aadhaar Number
✓ PAN Number
✓ Salary
✓ Bank Statement
✓ Credit Score

The application status interface allows users to view the current status of their submitted loan requests. Status information should reflect the application's actual processing and review workflow.

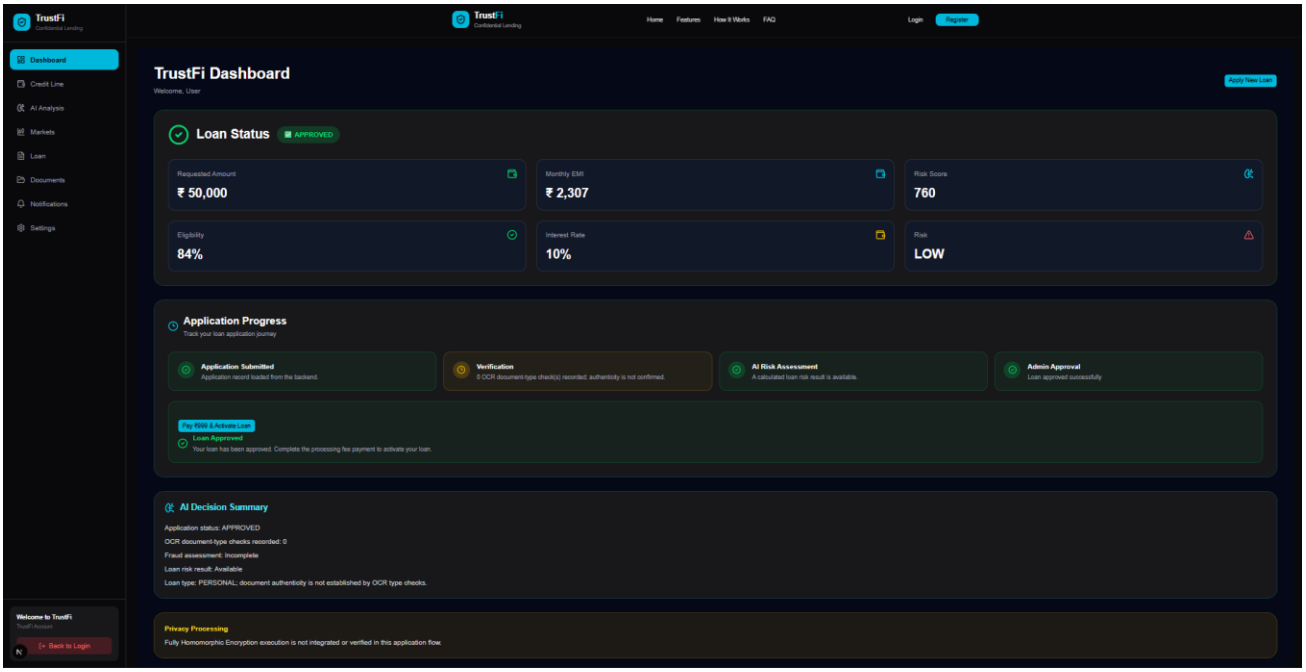
Clear status messages help applicants understand whether their requests are pending, under review, approved, rejected, or in another state supported by the implementation.

4.3.3 Eligibility or Assessment Results Screen

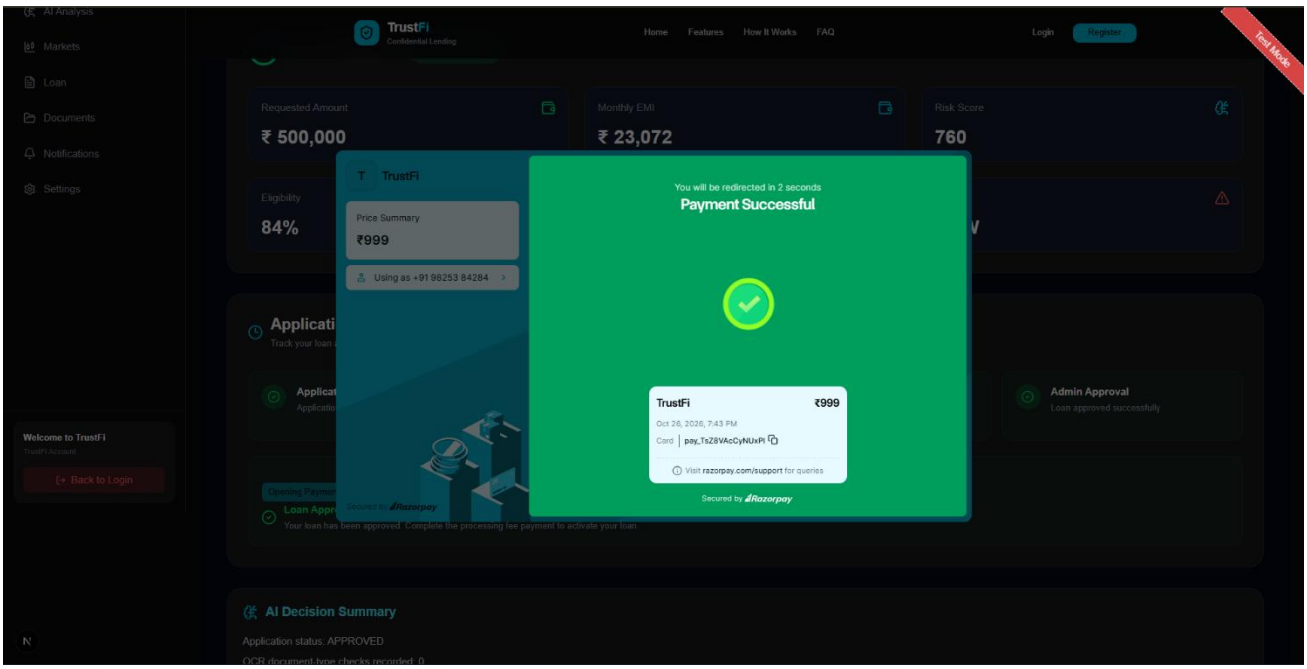
→Loan Application Submit Successful.



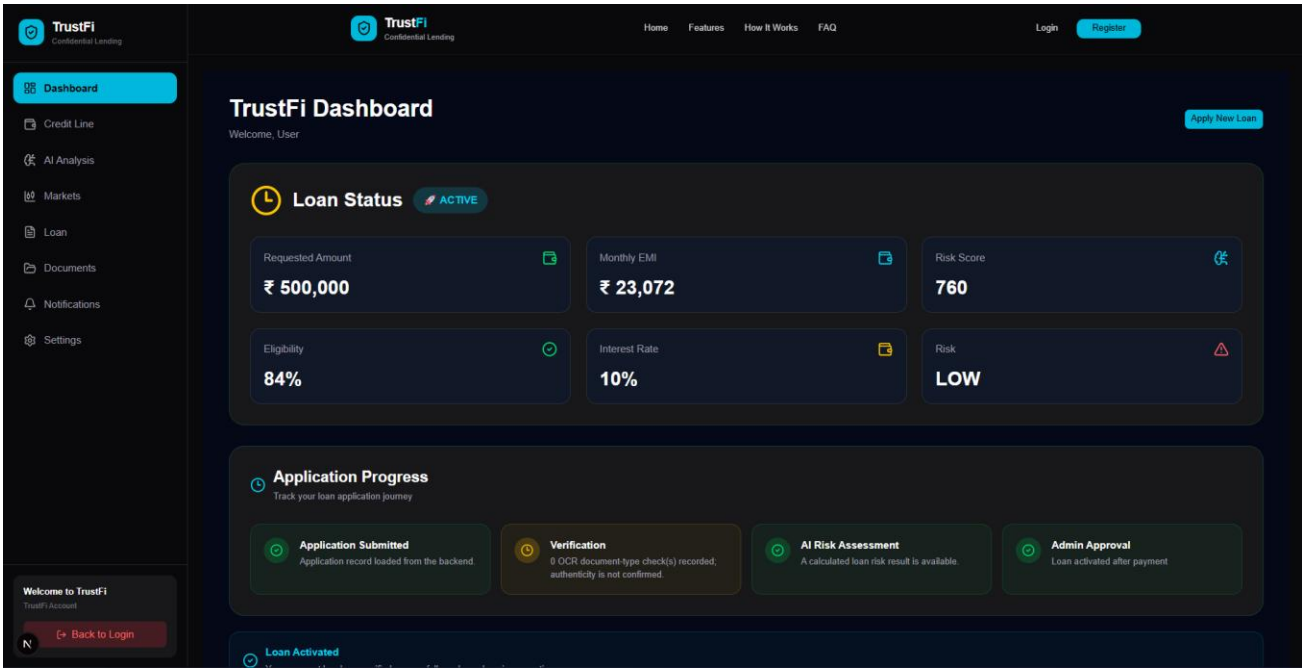
→After Approved loan dashboard.



→Pay 999 For Activate Loan



→Activation Loan

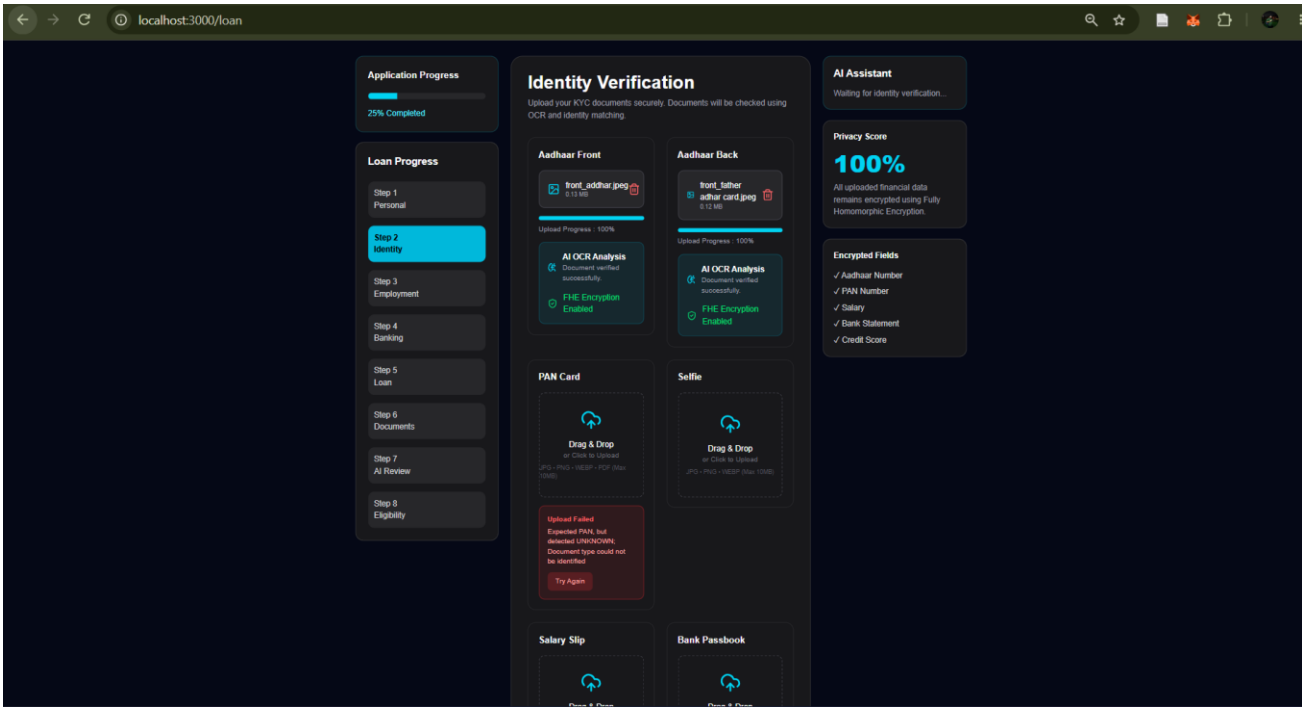


Where implemented, this interface presents the results generated by the platform's eligibility or risk-assessment process. The displayed information should correspond to the actual assessment logic and must not imply that an AI model has been trained or deployed unless that has been verified.

4.3.4 Administrative Review Screen

The administrative interface supports authorized personnel in examining submitted applications and performing available review actions. Depending on the implementation, it may present application details, document information, assessment results, and review controls.

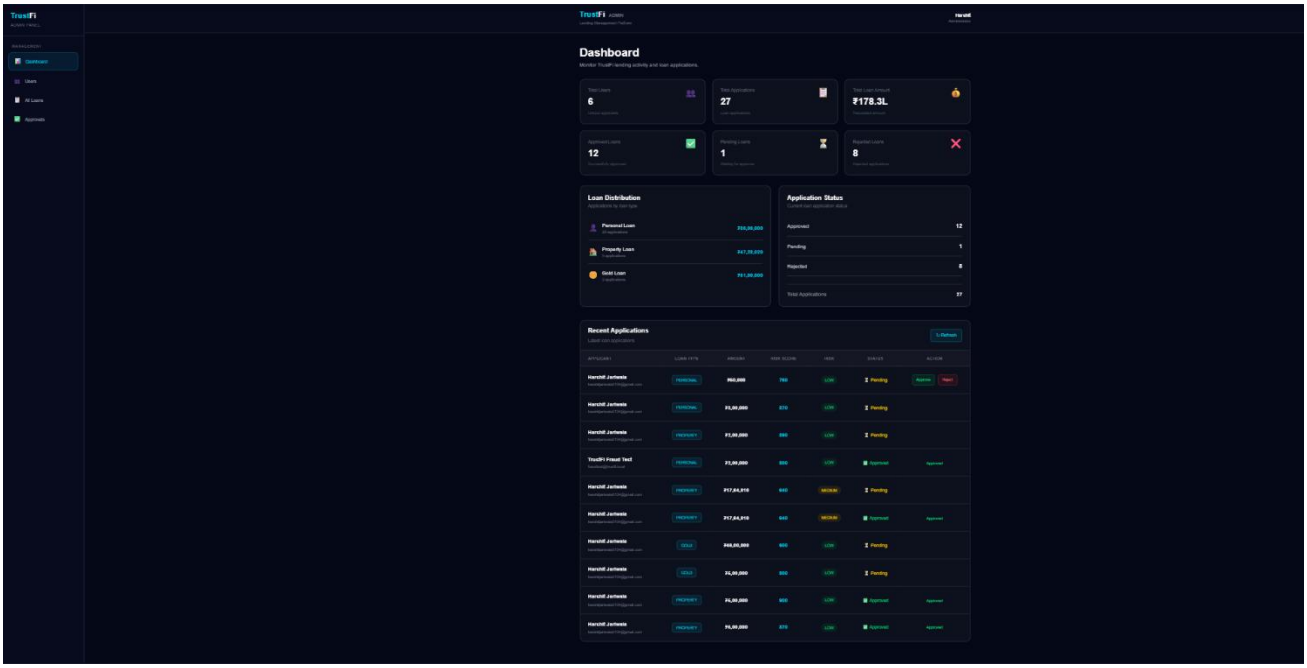
4.4 INPUT VALIDATION AND ERROR MESSAGES



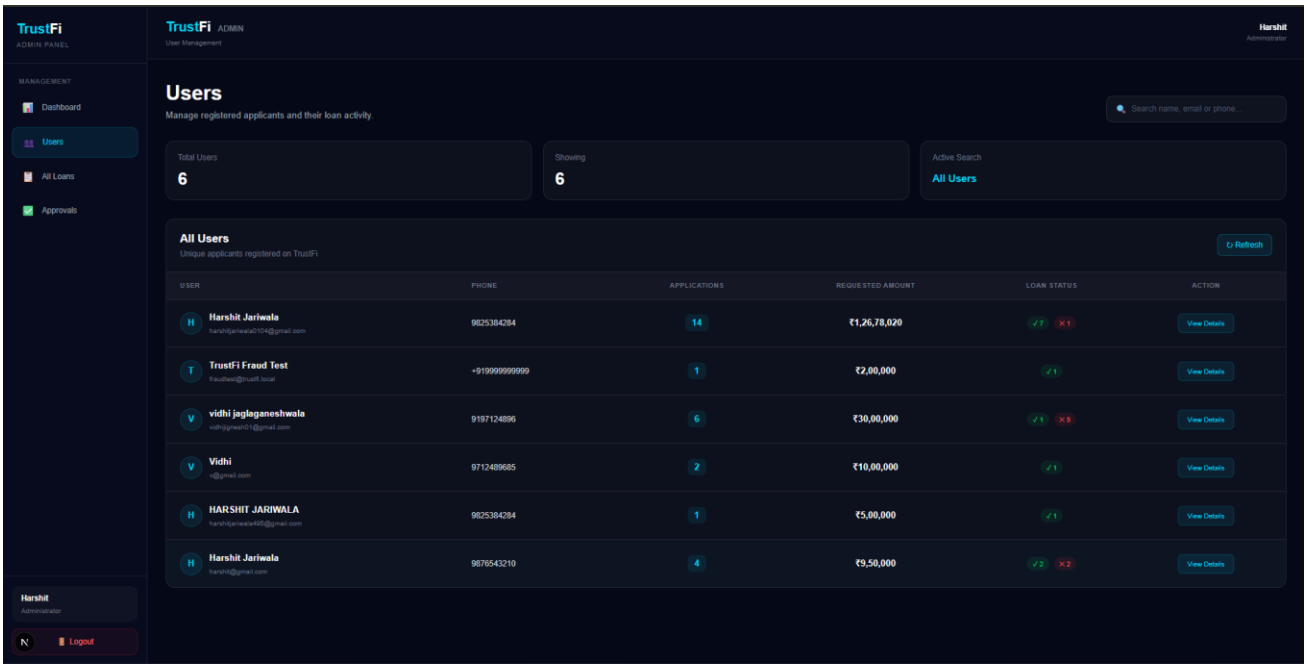
Input validation helps maintain the accuracy and consistency of information submitted to TrustFi. Required fields, supported file formats, authentication credentials, and other inputs should be validated according to the application's implemented rules.

When an error occurs, the interface should display a clear and understandable message. Examples include missing required information, invalid credentials, unsupported file formats, and failed submission requests. Only error messages and validation rules supported by the actual application should be documented as implemented features.

4.4.1 Admin Dashboard



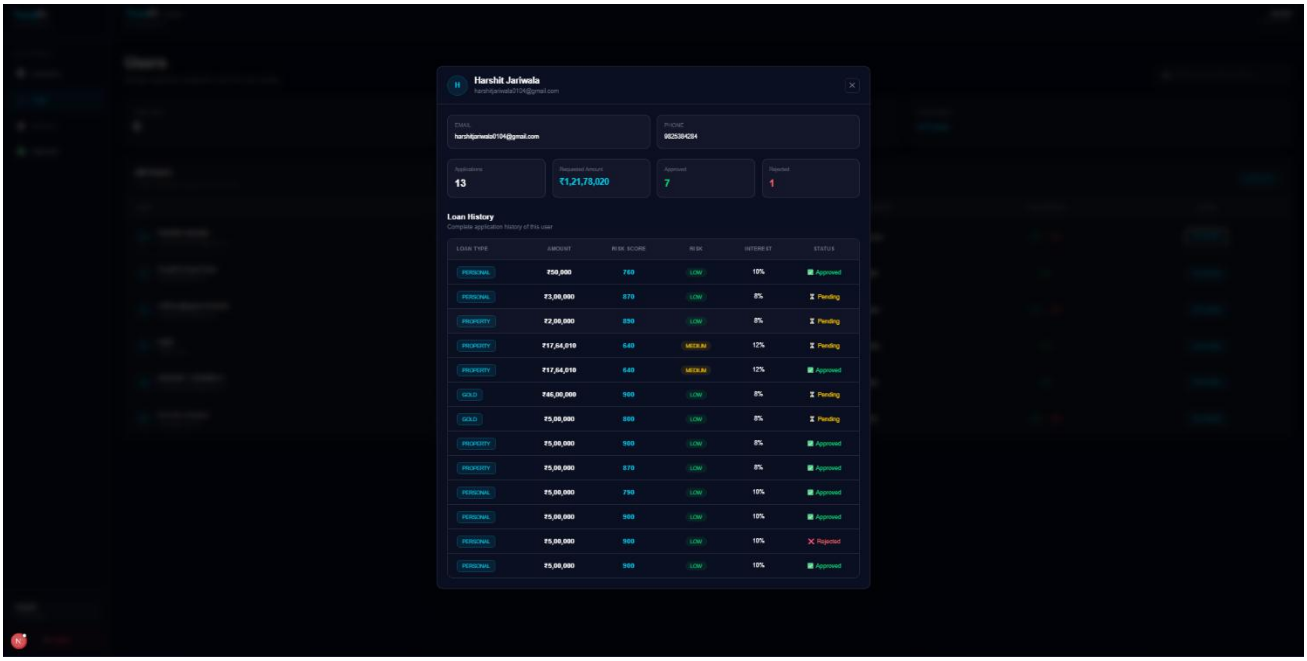
4.4.2 All User Management



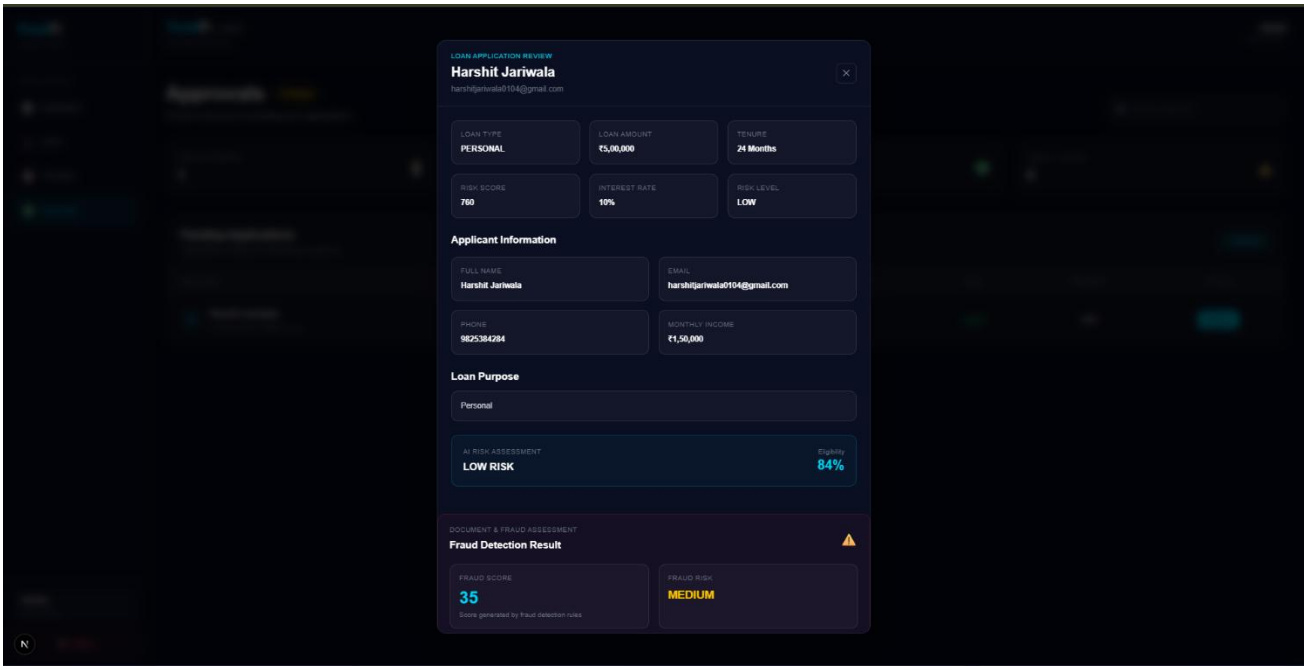
4.4.3 All Loan



4.4.3.1 Loan Details



4.4.3.2 Fruad Detaction



4.5 SUMMARY

The input and output interfaces form an important part of the TrustFi platform because they connect users with the system's application-processing workflow. Well-organized screens improve usability, support accurate data submission, and make application progress easier to understand. Actual screenshots should be included in the final report to demonstrate the implemented interfaces.

5. TEST PROCEDURES AND IMPLEMENTATION

5.1 INTRODUCTION

Software testing is an essential phase of the software development life cycle. It helps verify whether the application behaves according to its functional requirements and identifies errors that may affect usability, reliability, security, and data consistency.

Testing of the TrustFi platform focuses on the major application workflows, including user authentication, loan application submission, document upload, input validation, application status management, and administrative review. The scope of testing depends on the features implemented in the current project version.

5.2 IMPLEMENTATION OVERVIEW

TrustFi is developed as a web-based digital lending platform. The application follows a client-server approach in which the user interface communicates with backend services through APIs.

The frontend is based on Next.js, React, and TypeScript, with Zustand used for state management where required. The backend uses Node.js, Express, and TypeScript to handle application requests and business logic. Prisma ORM is used for database interactions, while JWT-based authentication is used for protected access where configured.

Tesseract OCR is associated with document text extraction in the project. The exact processing workflow and degree of integration should be verified against the current source code before being described as fully implemented.

The implementation is organized into functional components to improve maintainability and allow individual features to be developed and tested independently.

5.3 TESTING OBJECTIVES

The main objectives of testing are:

1. To verify that users can access the application through the implemented authentication workflow.
2. To validate the submission and handling of loan applications.
3. To check document-upload behavior and applicable file validations.
4. To verify that application information and status are displayed correctly.
5. To check access restrictions for protected resources and administrative functions.
6. To identify errors in API communication, data processing, and application navigation.
7. To verify that invalid inputs and unsuccessful requests are handled appropriately.

5.4 TYPES OF TESTING

5.4.1 Functional Testing

Functional testing checks whether each implemented feature performs its intended operation. It includes testing authentication, form submission, document handling, application tracking, and administrative actions where available.

5.4.2 Integration Testing

Integration testing examines communication between the frontend, backend APIs, authentication mechanisms, and database layer. It helps identify issues such as incorrect API responses, data inconsistencies, and failures in connected components.

5.4.3 Validation Testing

Validation testing checks whether the system accepts valid information and handles invalid or incomplete inputs appropriately. Examples include missing required fields, invalid login credentials, and unsupported document formats.

5.4.4 Security Testing

Security testing examines access control and the handling of sensitive information. Relevant checks include access to protected API routes, validation of authentication tokens, and prevention of unauthorized access to application records.

5.4.5 User Interface Testing

User interface testing checks navigation, form usability, error messages, layout consistency, and the display of application information across the supported interface.

5.5 TEST CASES

The following test cases provide a structured plan for evaluating the main TrustFi workflows. Execute each test against the current application and update the result column based on the observed behavior.

Test Case ID	**Test Scenario**	**Expected Result**	**Actual Result / Status**
TC-01	Open the application	The application loads without an unexpected error	To be tested
TC-02	Submit valid login credentials	The user is authenticated and receives the appropriate access	To be tested
TC-03	Submit invalid login credentials	Access is denied and an appropriate error is displayed	To be tested

TC-04	Submit a loan application with valid required fields	The application is processed according to the implemented workflow	To be tested
TC-05	Submit a loan application with missing required fields	Validation prevents an invalid submission or displays an appropriate error	To be tested
TC-06	Upload a supported document	The system handles the file according to the implemented upload workflow	To be tested
TC-07	Upload an unsupported document format	The system rejects the file or displays an appropriate validation message, according to its rules	To be tested
TC-08	View application status	The displayed status corresponds to the stored application information	To be tested
TC-09	Access a protected API without valid authentication	The request is denied where authentication is required	To be tested
TC-10	Perform an administrative review action	The action is processed only when authorized and supported by the application	To be tested

5.6 TESTING PROCEDURE

The testing procedure begins by starting the frontend and backend services with the required environment variables and database configuration. The application is then opened in a browser, and each test case is executed using appropriate valid and invalid inputs.

The tester records the expected behavior, observed behavior, and final status for every test case. Any difference between expected and actual results is investigated, corrected where possible, and tested again.

Testing should also include verification of relevant API responses and database changes where access to these components is available. Sensitive information should not be exposed in screenshots or testing records.

5.7 IMPLEMENTATION AND TESTING SUMMARY

Testing provides a systematic method for evaluating the reliability and correctness of TrustFi. The final testing summary should be prepared after executing the listed test cases against the actual project.

The report should include genuine test outcomes and supporting screenshots where appropriate. Features that have not been tested or are not fully implemented should be identified clearly rather than reported as successfully verified.

6. SAMPLE CODE

6.1 INTRODUCTION

This section presents the important code components of the TrustFi: AI-Assisted and Privacy-Aware Digital Lending Platform. The project focuses on providing a user interface for the platform's landing page and loan application workflow.

The sample code section is intended to explain the implementation of the major components, including the landing page, loan application form, form validation, application submission, and related backend or database operations where applicable.

The code listings included in this section should be taken from the current version of the project source code. Only features that are implemented in the submitted project should be documented as completed functionality.

6.2 LANDING PAGE IMPLEMENTATION

The landing page serves as the main entry point of the TrustFi platform. It introduces the purpose of the application and provides navigation to the available features.

The interface is designed to communicate the platform's digital lending workflow and guide users toward the loan application process. Its layout, navigation elements, and displayed information depend on the current frontend implementation.

Code Listing 6.1: Landing Page Component

```
import Navbar from "@/components/layout/Navbar";

import HeroSection from "@/components/landing/HeroSection";

import TrustedBySection from "@/components/landing/TrustedBySection";

import FeaturesSection from "@/components/landing/FeaturesSection";

import HowItWorksSection from "@/components/landing/HowItWorksSection";

import ArchitectureSection from "@/components/landing/ArchitectureSection";

import CTASection from "@/components/landing/CTASection";

import Footer from "@/components/layout/Footer";


export default function HomePage() {

  return (

    <main className="bg-[#050816] text-white">
```

```
      <Navbar />
      <HeroSection />
      <TrustedBySection />
      <FeaturesSection />
      <HowItWorksSection />
      <ArchitectureSection />
      <CTASection />
      <Footer />
    </main>
  );
}
```

The code listing should be followed by a brief explanation of the component structure, the main user interface elements, and the navigation actions supported by the landing page.

6.3 LOAN APPLICATION FORM

The loan application form enables users to provide the information required to initiate a loan request. Depending on the current implementation, the form may collect applicant details, loan-related information, and supporting information required for further processing.

The application form should validate the required inputs and guide users through the available steps. The exact number of steps and the fields displayed must correspond to the current version of the project.

Code Listing 6.2: Loan Application Form Component

```
const handleLoanTypeChange = (value: string) => {
  updateLoan({
    loanType: value as LoanType,
    purpose: "",
    // Reset loan-specific details when changing loan type
    propertyType: "",
    propertyValue: 0,
    propertyAddress: "",
    ownershipStatus: "",
```

```
    goldWeight: 0,
    goldPurity: "",
    goldValue: 0,
    goldOwnership: "",
  });
};
```

```
/* ---------------- CREDIT CALCULATION ---------------- */
  const result = useMemo(() => {
    return calculateCredit({
      monthlyIncome,
      loanAmount: amount,
      months,
      occupation,
      documentsVerified: true,
      bankVerified: true,
    });
  }, [
    monthlyIncome,
    occupation,
    amount,
    months,
  ]);
  const emi = result.emi;
  const risk = result.risk;
  const creditLimit = result.eligibleAmount;
```

```
/* ---------------- CONTINUE ---------------- */
const handleContinue = () => {
  if (!loanType || !purpose) {
    return;
  }
  if (amount <= 0 || months <= 0) {
    return;
  }
  // Property Loan validation
  if (loanType === "PROPERTY") {
    if (
      !loan.propertyType ||
      loan.propertyValue <= 0 ||
      !loan.propertyAddress ||
      !loan.ownershipStatus
    ) { return;  }}
```

The accompanying explanation should describe how the form captures user input, manages form state, validates required fields, and allows the user to proceed through the application workflow.

6.4 FORM VALIDATION AND STEP MANAGEMENT

Where a multi-step loan application is implemented, form state management helps preserve the information entered by the user while moving between steps.

Validation ensures that required information is provided before the application proceeds to the next step or is submitted. Appropriate validation messages should help users identify and correct invalid or missing information.

Code Listing 6.3: Actual Form Validation or Step Navigation Logic

---

```
export function calculateRisk(
  input: RiskInput
```

```
): RiskResult {
  let score = 300;
  let fraudScore = input.fraudScore ?? 0;
  const riskReasons: string[] = [];
  const fraudFlags: string[] = [];
---------------------------------------------------------------------------------------------------------------
/* ---------------- CREDIT RISK ---------------- */
  let risk: "LOW" | "MEDIUM" | "HIGH";
  let interestRate: number;
  let approved: boolean;
  if (score >= 800) {
    risk = "LOW";
    interestRate = 8;
    approved = true;
    riskReasons.push(
      "Overall credit risk is low based on the calculated score."
    );} else if (score >= 700) {
    risk = "LOW";
    interestRate = 10;
    approved = true;
    riskReasons.push(
      "Overall credit risk is low with acceptable repayment indicators."
    ); } else if (score >= 600) {
    risk = "MEDIUM";
    interestRate = 12;
    approved = true;
    riskReasons.push(
      "Overall credit risk is medium and requires additional consideration."
    ); } else {
    risk = "HIGH";
```

```
    interestRate = 16;
    approved = false;
    riskReasons.push(
      "Overall credit risk is high based on the calculated score."
    );  }
```

```
/* ---------------- FRAUD FLAGS ---------------- */
  if (input.fraudFlags && input.fraudFlags.length > 0) {
    for (const flag of input.fraudFlags) {
      if (
        typeof flag === "string" &&
        flag.trim().length > 0
      ) {
        fraudFlags.push(flag.trim());
      }}}
```

The code explanation should describe the actual validation conditions, navigation behavior, and handling of incomplete information.

6.5 LOAN APPLICATION SUBMISSION

The submission component handles the action performed when a user completes the loan application. Depending on the implementation, the application may send the entered information to a backend API, save the information through a database layer, or use another configured processing mechanism.

The final report should clearly describe the actual submission flow and the response presented to the user. A successful submission message should be documented only when that behavior has been verified.

Code Listing 6.4: Loan Application Submission Handler

```
import { Router } from "express";
import { applyLoan,
  getLoans,
  approveLoan,
```

```
  rejectLoan,
  getLoanByEmail,
  calculateLoanRisk,
} from "../controllers/loan.controller";
const router = Router();
router.post("/apply", applyLoan);
router.get("/", getLoans);
router.post("/risk", calculateLoanRisk);
router.get("/:email", getLoanByEmail);
router.patch("/approve/:id", approveLoan);
router.patch("/reject/:id", rejectLoan);
export default router;
```

6.6 BACKEND API AND DATABASE OPERATIONS

If the current version includes backend APIs and database persistence for loan applications, this subsection should document the relevant implementation.

The backend code may handle incoming application requests, validate submitted information, and perform the database operations defined by the project. The actual API route, request fields, response format, and database models must be verified from the source code.

Code Listing 6.5: Actual Loan Application API or Database Operation

---

```
/* ============================================================ */
/* CALCULATE LOAN RISK */
/* ============================================================ */
export const calculateLoanRisk = async (
  req: Request,
  res: Response
) => {
  try {  const {
     monthlyIncome,
```

```
  loanAmount,
  loanMonths,
  creditScore,
  occupation,
  documentsVerified,
  bankVerified,
} = req.body;
if (
  monthlyIncome === undefined ||
  loanAmount === undefined ||
  loanMonths === undefined ||
  !occupation ||
  documentsVerified === undefined ||
  bankVerified === undefined
) {
  return res.status(400).json({
    success: false,
    message: "Missing required risk assessment fields.",
  }); }
const income = Number(monthlyIncome);
const amount = Number(loanAmount);
const months = Number(loanMonths);
const suppliedCreditScore = Number(creditScore || 0);
if (!Number.isFinite(income) ||
  income <= 0 ||
  !Number.isFinite(amount) ||
  amount <= 0 ||
  !Number.isInteger(months) ||
  months <= 0 ||
  !Number.isFinite(suppliedCreditScore)
```

```
) {  return res.status(400).json({
    success: false,
    message:
      "Income, loan amount, duration, or credit score is invalid.",
  }); }
if (typeof documentsVerified !== "boolean" ||
  typeof bankVerified !== "boolean"
) { return res.status(400).json({
    success: false,
    message:
      "Document and bank verification values must be boolean.",
  });}
const fraudInputs = normalizeFraudInputs(req.body);
if (fraudInputs.fraudScore !== undefined &&
  (fraudInputs.fraudScore < 0 ||
    fraudInputs.fraudScore > 100
  )) {
  return res.status(400).json({
    success: false,
    message: "Fraud score must be between 0 and 100.",
  });}
const result = calculateRisk({
  monthlyIncome: income,
  loanAmount: amount,
  loanMonths: months,
  creditScore: suppliedCreditScore,
  occupation: String(occupation),
  documentsVerified,
  bankVerified,
  ...fraudInputs,
```

```
    });
    return res.json({
      success: true,
      risk: result,
    });} catch (error: unknown) {
    console.error("Risk Calculation Error:", error);
    return res.status(500).json({
      success: false,
      message: "Risk calculation failed.",
    });
 }};
```

If application submission is currently handled only on the frontend, this subsection should be revised to describe the actual behavior instead of claiming that backend storage is implemented.

6.7 ERROR HANDLING

Error handling helps the application respond appropriately to invalid inputs, failed requests, and unexpected processing problems. The current implementation should be examined to identify how validation errors and submission failures are handled.

Code Listing 6.6: Actual Error-Handling Logic

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
```

```
model User {
  id        String           @id @default(cuid())
  name      String
  email     String           @unique
  phone     String           @unique
  createdAt DateTime         @default(now())
```

```
  updatedAt DateTime          @updatedAt
  loans     LoanApplication[]
  otps      UserOTP[]
}

-----------------------------------------------------------------------------------------------------------------

enum LoanType {
  PERSONAL
  PROPERTY
  GOLD
}}

-----------------------------------------------------------------------------------------------------------------
```

The accompanying explanation should describe the actual error conditions and how the application communicates them to the user.

6.8 SUMMARY

The sample code section documents the principal implementation components of the current TrustFi project. The landing page and loan application workflow form the primary focus of this section, while backend APIs, database operations, validation, and error handling are included according to their availability in the submitted source code.

Including verified source-code excerpts and clear explanations helps demonstrate the technical implementation of the project and establishes consistency between the report and the application being submitted.

7. LIMITATIONS OF THE SYSTEM

7.1 INTRODUCTION

TrustFi is designed to support digital lending workflows through a web-based platform. Although the system aims to simplify loan application submission, document handling, and application tracking, certain technical and operational limitations may affect its functionality and scalability.

Understanding these limitations is important for evaluating the current implementation and identifying areas for future improvement.

7.2 TECHNICAL LIMITATIONS

7.2.1 Dependence on Internet Connectivity

TrustFi is a web-based application that depends on network connectivity for communication between the frontend and backend services. Unstable internet access or server unavailability may interrupt application submission, document uploads, and retrieval of application information.

7.2.2 Document Quality and OCR Accuracy

OCR-based text extraction depends on the quality, clarity, orientation, and format of uploaded documents. Blurred images, handwritten content, unusual layouts, and low-resolution scans may reduce extraction accuracy. Extracted information may therefore require validation before being used in further processing.

7.2.3 Dependence on Configured Services

The application relies on correctly configured frontend and backend environments, database connectivity, authentication settings, and any external services used by the implementation. Incorrect configuration or service failure may affect application functionality.

7.2.4 Limited Assessment Reliability

Eligibility and risk-assessment results depend on the rules, input data, and algorithms actually implemented in the system. Such results should not be treated as definitive lending decisions without appropriate validation and review. If a trained machine-learning model is not implemented, the system should not be represented as providing model-based predictions.

7.3 SECURITY AND PRIVACY LIMITATIONS

7.3.1 Protection of Sensitive Information

Loan applications may contain personal, financial, and identity-related information. Appropriate authentication, authorization, secure storage, data minimization, and access controls are necessary to protect this information. The effectiveness of these measures depends on the actual implementation and configuration.

7.3.2 Authentication and Access Control

JWT-based authentication can help protect restricted resources, but its effectiveness depends on correct token verification, secure secret management, appropriate token expiry, and authorization checks. Authentication alone does not guarantee complete application security.

7.3.3 Data Privacy and Compliance

The handling of applicant information must follow applicable privacy requirements and organizational policies. Additional privacy controls, retention rules, audit mechanisms, and security reviews may be required before the platform can be used in a real lending environment.

7.4 OPERATIONAL LIMITATIONS

7.4.1 Dependence on Accurate User Input

Incorrect, incomplete, or misleading information submitted by applicants may affect the quality of the assessment process. Input validation can reduce certain errors but cannot independently guarantee the truthfulness of all submitted information.

7.4.2 Administrative Review Requirements

Certain applications may require manual verification of documents, applicant information, or assessment results. The extent of administrative involvement depends on the implemented workflow and the requirements of the lending institution.

7.4.3 External Integration Constraints

Where external services, such as payment gateways or third-party APIs, are used, their availability, configuration, access permissions, and service limitations may affect related workflows. Features that are not fully integrated or tested should not be considered production-ready.

7.5 SCALABILITY AND PERFORMANCE

The performance of TrustFi depends on the efficiency of its application code, database queries, file-processing operations, server resources, and network conditions. Increased numbers of users or large document uploads may require additional performance testing and infrastructure improvements.

Load testing, database optimization, monitoring, and suitable deployment architecture would help evaluate whether the platform can support larger workloads reliably.

7.6 SUMMARY

The current TrustFi implementation provides a foundation for exploring digital loan application workflows and related software technologies. However, production use would require further validation of functionality, security, privacy, performance, and external integrations.

These limitations identify opportunities for future development and should be assessed against the actual features and test results of the project.

8. FUTURE SCOPE

8.1 INTRODUCTION

TrustFi provides a foundation for digitizing important stages of the lending application process. The platform can be extended with additional capabilities to improve automation, security, transparency, scalability, and user experience.

Future development should focus on improving the reliability of existing workflows while introducing new features after appropriate technical evaluation and testing.

8.2 ADVANCED AI-BASED CREDIT ASSESSMENT

A future version of TrustFi could incorporate validated machine-learning models to support credit-risk assessment and eligibility analysis. Such models could analyze suitable financial and application-related information to identify patterns and generate assessment indicators.

Before deployment, these models would require reliable training data, performance evaluation, bias testing, explainability measures, and regular monitoring. Automated assessments should support informed decision-making rather than replace appropriate human oversight.

8.3 ENHANCED DOCUMENT PROCESSING

The document-processing module could be improved through more advanced OCR techniques, document classification, field extraction, and automated validation. These improvements could reduce manual data entry and help identify missing or inconsistent information.

Additional validation mechanisms could compare extracted information with applicant-provided details and flag discrepancies for authorized review.

8.4 PRIVACY-ENHANCING TECHNOLOGIES

Future development could investigate privacy-enhancing technologies such as data minimization, encryption, secure computation, and federated learning where appropriate.

Fully Homomorphic Encryption (FHE) could be explored for selected computations on encrypted data, subject to performance and implementation feasibility. Such capabilities would require careful design and testing before being presented as part of the operational system.

8.5 BLOCKCHAIN-BASED AUDITABILITY

A future version could investigate blockchain technology for maintaining tamper-evident records of selected application events, consent records, or administrative actions.

Sensitive applicant documents and personal information should not be placed directly on a public blockchain. A suitable design would need to evaluate privacy, access control, transaction costs, data-retention requirements, and the risks associated with permanently recorded information.

8.6 IMPROVED USER NOTIFICATIONS

The platform could be extended to provide notifications about application submission, requests for additional documents, review progress, and final status updates.

Notifications through email, SMS, or other supported channels could help users remain informed. Such integrations would require appropriate configuration, user consent where applicable, and reliable handling of delivery failures.

8.7 PAYMENT AND EXTERNAL SERVICE INTEGRATION

Where required by the lending workflow, TrustFi could integrate with verified external services for payment processing, identity verification, or other authorized operations. Each integration would require appropriate security controls, error handling, and testing.

Any payment-related functionality should clearly distinguish application fees or other supported transactions from loan disbursement and repayment services.

8.8 SCALABILITY AND CLOUD DEPLOYMENT

Future development could include deployment on scalable cloud infrastructure, automated deployment pipelines, centralized logging, application monitoring, database optimization, and reliable backup procedures.

Performance testing and capacity planning would help determine the infrastructure required to support increasing numbers of users and document-processing requests.

8.9 ACCESSIBILITY AND USER EXPERIENCE

The user interface could be improved through responsive layouts, accessible form controls, clearer validation messages, multilingual support, and simplified navigation. These improvements could make the platform easier to use across different devices and for users with varied technical experience.

8.10 SUMMARY

The future scope of TrustFi includes opportunities to enhance document automation, assessment capabilities, privacy protection, auditability, external integrations, and infrastructure scalability. These improvements should be prioritized according to user requirements, technical feasibility, security considerations, and available resources.

The proposed enhancements represent potential future work and should not be interpreted as features already implemented in the current version.

9. CONCLUSION

TrustFi: AI-Assisted and Privacy-Aware Digital Lending Platform is a web-based project focused on simplifying and organizing the digital loan application process. The project explores how modern web technologies can support applicant information management, document submission, application tracking, and loan-related assessment workflows within a unified platform.

The application uses technologies such as Next.js, React, TypeScript, Node.js, Express, and Prisma ORM to structure the frontend, backend services, and database interactions. JWT-based authentication and OCR-related document processing are also relevant components of the system, subject to their configuration and implementation in the current project version.

The development of TrustFi provides practical experience in full-stack web application development, API communication, authentication, data management, input validation, and software testing. It also highlights the importance of protecting sensitive applicant information and maintaining reliable application workflows.

The project demonstrates the potential of digital platforms to make loan application processes more organized and accessible. However, the reliability of eligibility assessments, the effectiveness of security controls, and the completeness of external integrations depend on the actual implementation and testing performed.

Further development could introduce validated AI-based assessment techniques, enhanced document processing, privacy-enhancing technologies, improved auditability, and scalable deployment infrastructure. These enhancements would require careful evaluation before being used in a real lending environment.

In conclusion, TrustFi establishes a foundation for exploring technology-assisted digital lending workflows. Its continued improvement can focus on usability, security, transparency, and reliability while ensuring that implemented features are clearly distinguished from proposed enhancements.

10. REFERENCES

The following references provide technical documentation and learning resources related to the technologies and concepts used in the development of the TrustFi platform.

1. **Next.js Documentation.** Official documentation for the Next.js web development framework. Available at: https://nextjs.org/docs
2. **React Documentation.** Official documentation for building user interfaces using React. Available at: https://react.dev/
3. **TypeScript Documentation.** Official documentation for the TypeScript programming language. Available at: https://www.typescriptlang.org/docs/
4. **Node.js Documentation.** Official documentation for the Node.js JavaScript runtime environment. Available at: https://nodejs.org/docs/
5. **Express.js Documentation.** Official documentation for building web applications and APIs using Express. Available at: https://expressjs.com/
6. **Prisma Documentation.** Official documentation for database access and ORM functionality. Available at: https://www.prisma.io/docs
7. **Tesseract OCR Documentation.** Documentation and resources for Optical Character Recognition using Tesseract. Available at: https://github.com/tesseract-ocr/tesseract
8. **JSON Web Token (JWT).** Official website providing information about JSON Web Tokens and their use in authentication. Available at: https://jwt.io/
9. **Zustand Documentation.** Documentation for state management in React applications. Available at: https://zustand.docs.pmnd.rs/
10. **Razorpay Documentation.** Official documentation for payment gateway integration, where applicable to the project. Available at: https://razorpay.com/docs/
11. **Project Source Code.** TrustFi project repository and associated source files, used as the primary reference for the actual implementation. Repository: [Insert the project's GitHub repository URL, if publicly accessible].